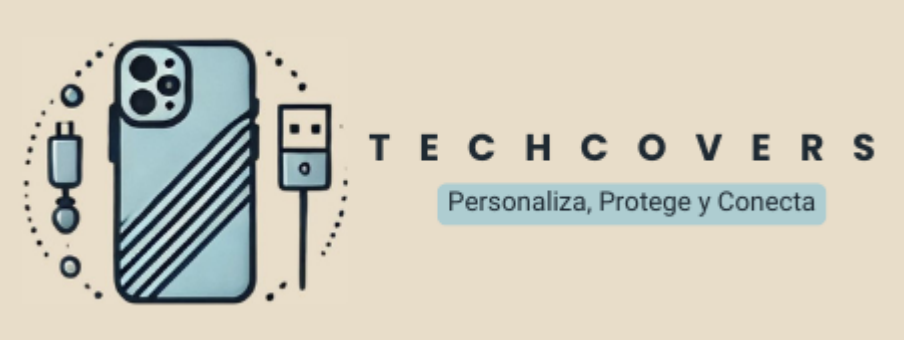
TechCovers



Memoria técnica del proyecto

Plataforma web de consulta y reserva de productos y accesorios para móviles

Daniela Zabala Ovalle

Módulo Python

CEI Centro de Estudios Innovación

Octubre de 2026

Enlaces e información:
https://github.com/Daniela-Zabala/daniela_zabala_proyecto_final.
https://daniela-zabala-proyecto-final.vercel.app/

Usuario	Contraseña	Permiso
creador	123456	administrador
profesor	python2026	
prueba	123456prueba	

Índice

1.Introducción 3
2. Objetivos del proyecto 4
3. Tecnologías utilizadas 4
5. Diseño de la base de datos 5
6. Inicialización del proyecto y configuración del entorno 6
7. Conexión con Django 7
8. Control de versiones con GitHub 8
9. Puesta en funcionamiento online 9
11. Maquetación e interfaz de TechCovers 10
12. Maquetación de páginas 10
13. Gestión de usuarios 12
14. Creación de los modelos **13**
15. Cambio de la base de datos a Neon 14
16. Comprobación de la nueva base de datos 15
17. Desarrollo de Formularios y Vistas 16
18.Comprobación de las categorías 19
19. Catálogo de Productos 19
20. Plantillas comunes 22
21. Registro e inicio de sesión 23
22.Creación de las plantilla de Registro e inicio de sesión 24
23. Conexión del registro con Django 25
24. Configuración de la ruta de registro 25
25. Comprobación del registro 26
26. Configuración Inicio de sesión y cierre 27
27. Redirecciones 28
28. Menú adaptado al estado del usuario 28
29.Creación de la sección "Mis reservas" 29
30. Creación de las páginas de error 404 y 500 30
31. Control de permisos para la creación de productos 31
32. Gestión y modificación de productos 32
33. Unificación de las plantillas y mejora de la navegación 33
34. Gestión de Reservas 35
35. Pruebas Automáticas con Django. 36
36. Docstrings 37
37. Rediseño página de Inicio. 39
38. Logs. 39
39. Pseudocódigo 40
40.Configuración de Debug. 41
41. Comprobaciones Finales. 42
42. Recorrido por el funcionamiento final de TechCovers 4
43. Mejoras Futuras TechCovers 48

La presente memoria documenta el desarrollo de TechCovers desde sus primeras versiones hasta las últimas mejoras incorporadas. Su estructura sigue, en gran medida, el orden cronológico del proceso de desarrollo, por lo que algunos apartados y capturas muestran versiones anteriores de la aplicación, ya que las funcionalidades se fueron implementando progresivamente. Los apartados posteriores recogen las nuevas funcionalidades y mejoras añadidas, manteniendo el recorrido original para reflejar la evolución del proyecto. Finalmente, los últimos apartados muestran el funcionamiento final de TechCovers y permiten conocer el resultado alcanzado tras completar su desarrollo.

1.Introducción

TechCovers es una plataforma web creada para la consulta y reserva de productos y accesorios para dispositivos móviles.

La plataforma permite a los usuarios consultar un catálogo de productos organizados por diferentes categorías, visualizar la información de cada producto y realizar reservas de aquellos que sean de su interés.

El objetivo principal es facilitar la consulta y el proceso de reserva de accesorios para móviles mediante una plataforma web sencilla e intuitiva. TechCovers no se plantea como una tienda online, ya que no se realizan compras ni pagos a través de la plataforma (en principio).

Para el desarrollo del proyecto se utiliza **Python y Django**, junto con una base de datos **SQLite (esto cambiará más adelante)**, siguiendo una estructura que permite organizar la información de los productos, categorías, usuarios y reservas.



2. Objetivos del proyecto

El objetivo principal es desarrollar una plataforma web funcional que permita gestionar un catálogo de accesorios para móviles y realizar reservas de productos.

Los objetivos específicos son:

- Crear un catálogo de productos organizado por categorías.
- Permitir consultar la información de cada producto.
- Gestionar los productos disponibles en la plataforma.
- Gestionar las categorías de productos.
- Permitir a los usuarios registrarse e iniciar sesión
- Permitir a los usuarios realizar reservas.
- Permitir a los usuarios consultar sus reservas.
- Utilizar una base de datos para almacenar la información de la aplicación.
- Crear una interfaz web sencilla y fácil de utilizar.

3. Tecnologías utilizadas

Para desarrollar TechCovers se utilizan las siguientes tecnologías:

- **Python** como lenguaje principal de programación.
- **Django** como framework para el desarrollo de la plataforma web.
- **SQLite** como sistema de base de datos.
- **HTML y CSS** para la creación y diseño de la interfaz.
- **Git y GitHub** para el control de versiones y almacenamiento del código.
- **Draw.io** para la representación y diseño de la estructura de la base de datos.
- **Vercel:** Es una plataforma en la nube para desplegar proyectos de manera online.
- **Neon:** plataforma utilizada para gestionar la base de datos en el entorno online.
- **Visual Studio Code:** entorno utilizado para desarrollar y organizar el código del proyecto.

4. Estructura de la información

Para organizar la información de TechCovers se ha diseñado una base de datos formada por **cinco tablas principales**: Usuario, producto, reserva, categoría, producto-reserva.

Las tablas y sus relaciones son las siguientes:

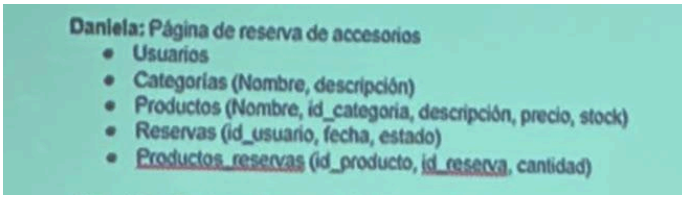
- **Usuario**: almacena la información de los usuarios registrados en la plataforma y es gestionado mediante el sistema de autenticación de Django.
- **Categoría**: permite organizar los productos según diferentes categorías y contiene el nombre y la descripción de cada una.
- **Producto**: almacena la información de los accesorios disponibles, incluyendo nombre, descripción, precio, stock y la categoría a la que pertenecen.

- **Reserva**: registra las reservas realizadas por los usuarios, incluyendo la fecha y el estado de cada reserva.
- **producto_reserva**: relaciona los productos con las reservas y permite indicar qué productos forman parte de cada reserva y qué cantidad se ha reservado.

Las relaciones establecidas son:

- Una **categoría** puede tener varios productos.
- Cada **producto** pertenece a una categoría.
- Un **usuario** puede realizar varias reservas.
- Cada **reserva** pertenece a un usuario.
- Una **reserva** puede contener varios productos.
- Un **producto** puede formar parte de diferentes reservas.
- La tabla **producto_reserva** permite establecer la relación entre productos y reservas y almacenar la cantidad reservada.

La estructura diseñada se puede representar mediante el siguiente esquema:

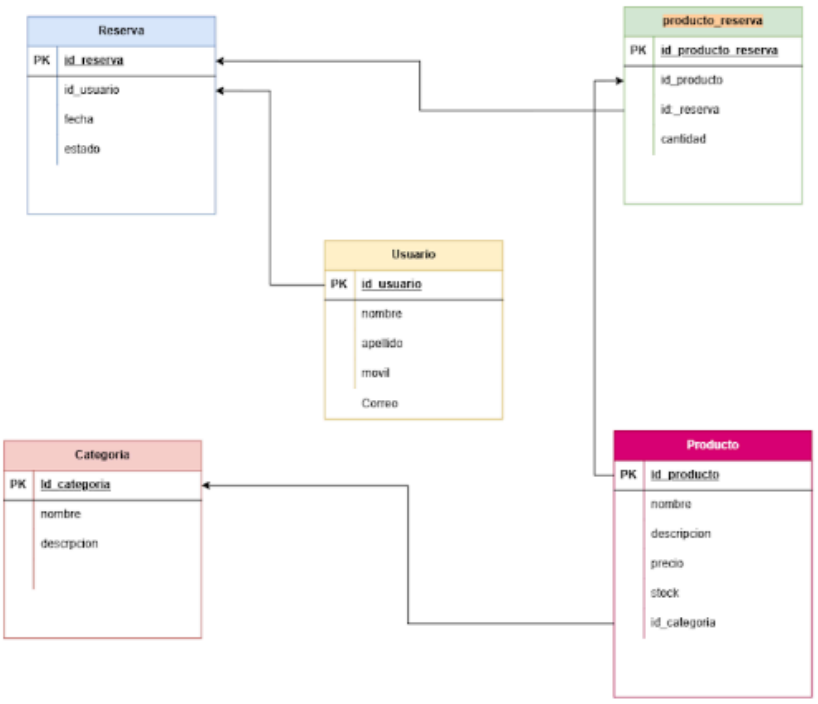


5. Diseño de la base de datos

Antes de comenzar con la implementación de la aplicación, se realizó el diseño de la estructura de la base de datos mediante un diagrama entidad-relación.

Este diseño permite visualizar las diferentes tablas que forman parte del proyecto y las relaciones existentes entre ellas.

La estructura se diseñó buscando que la información estuviera organizada y que las relaciones entre usuarios, productos, categorías y reservas fueran claras.



6. Inicialización del proyecto y configuración del entorno

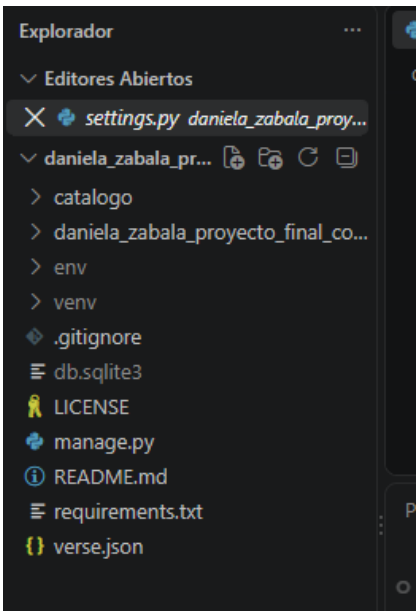
Una vez definida la estructura de la base de datos, se comenzó con la preparación del entorno de desarrollo en **Visual Studio Code**.

En primer lugar, se creó la estructura inicial del proyecto y se preparó el entorno necesario para trabajar con Python.

A continuación, se configuró el proyecto utilizando **Django**, que será el framework encargado de gestionar la aplicación web.

Durante esta fase también se organizó la estructura de carpetas y archivos necesaria para el desarrollo de TechCovers.

El objetivo de esta etapa fue disponer de un entorno de trabajo preparado y correctamente configurado antes de comenzar con el desarrollo de la aplicación.



7. Conexión con Django

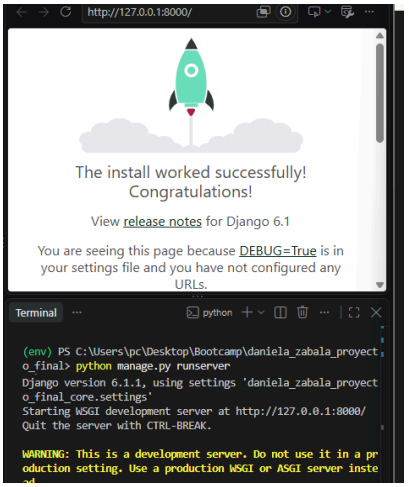
Una vez preparado el entorno de desarrollo, se realizó la configuración del proyecto utilizando Django como framework para el desarrollo de la plataforma web.

Django permite organizar la aplicación mediante diferentes componentes, entre ellos modelos, vistas y plantillas.

Después de realizar la configuración inicial, se ejecutó el servidor de desarrollo para comprobar que el proyecto funcionaba correctamente.

Esta primera comprobación permitió verificar que Django estaba correctamente instalado y que la aplicación podía ejecutarse desde el entorno local.

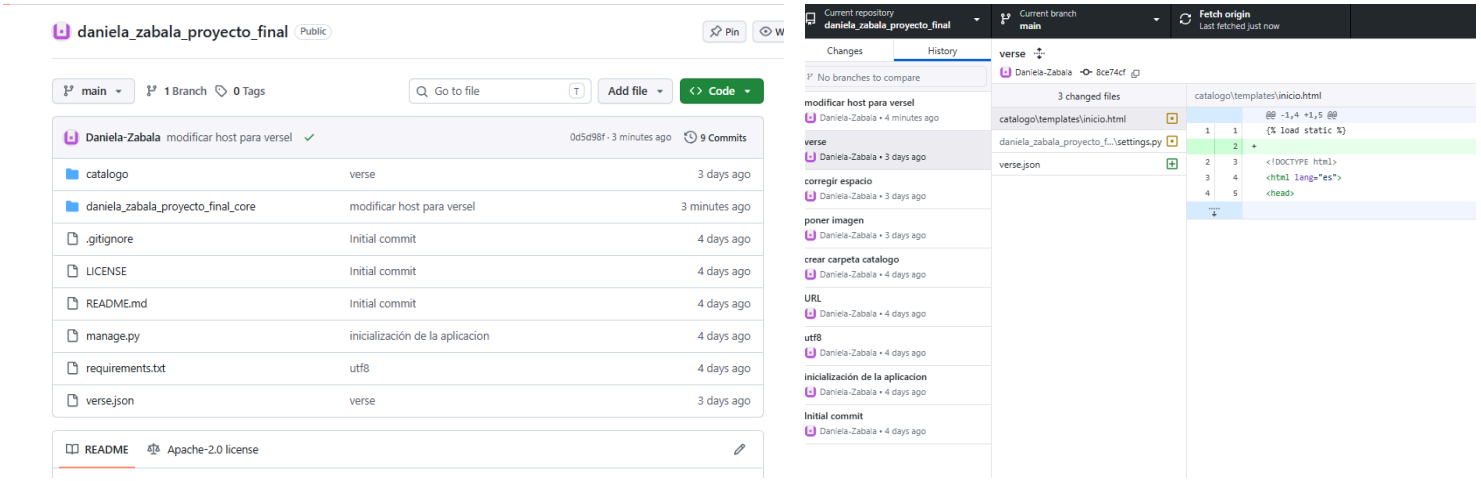
Posteriormente, la página inicial de Django fue sustituida por la página inicial diseñada específicamente para TechCovers



8. Control de versiones con GitHub

Una vez que el proyecto comenzó a funcionar correctamente, se subió a **GitHub** para disponer de una copia del código y poder llevar un control de los cambios realizados durante el desarrollo.

Para facilitar este proceso se utilizó **GitHub Desktop**, mediante el cual se fueron guardando las diferentes modificaciones realizadas en el proyecto.

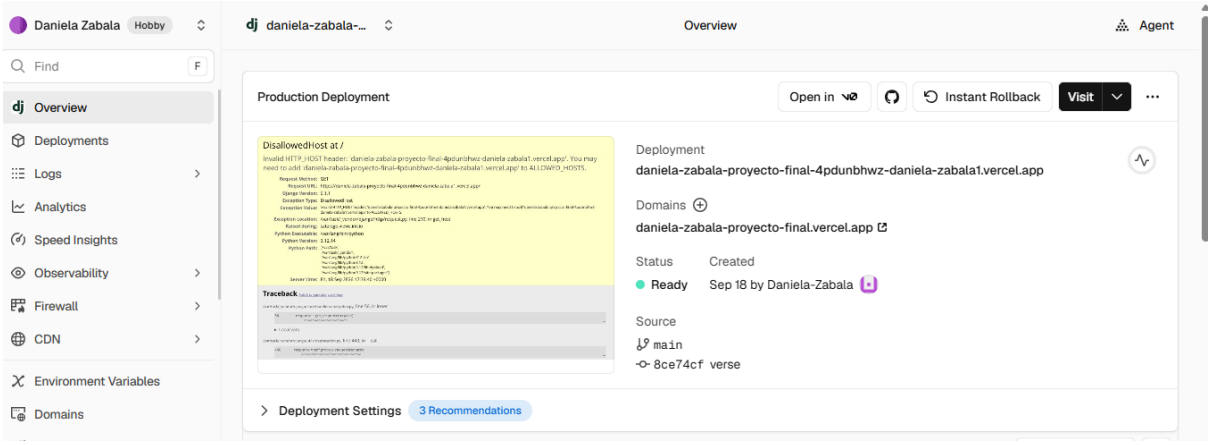


https://github.com/Daniela-Zabala/daniela_zabala_proyecto_final

9. Puesta en funcionamiento online

Después de comprobar que el proyecto funcionaba correctamente en el entorno de desarrollo, se realizó la conexión con **Vercel** para disponer de la aplicación en un entorno online.

Esta configuración permite comprobar que TechCovers podía ejecutarse fuera del entorno local y quedar preparada para ser accesible a través de Internet.



Más adelante volveremos a hacer una nueva conexión de la cual se dejará el enlace definitivo

10. Desarrollo de la aplicación.

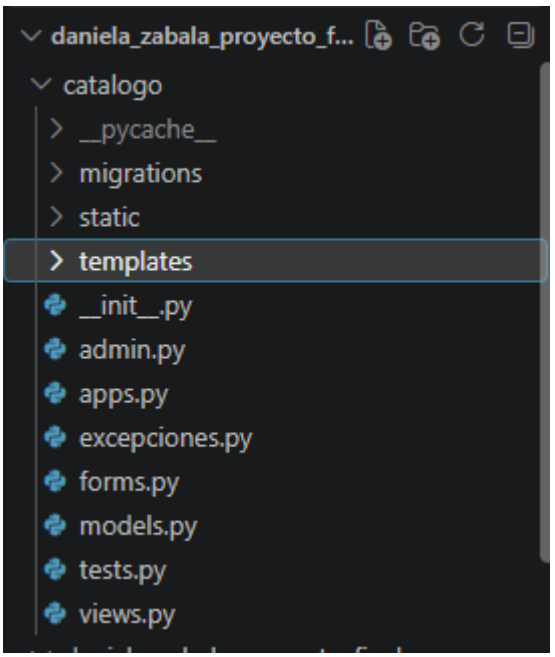
El proyecto se desarrolla utilizando Django, siguiendo su estructura basada en modelos, vistas y plantillas.

Los modelos (models.py) representan la información que se almacena en la base de datos.

Las vistas (views.py) se encargan de recibir las peticiones del usuario, trabajar con la información necesaria y devolver la respuesta correspondiente.

Las plantillas HTML permiten construir la parte visual de la plataforma y mostrar la información al usuario. (dentro de catalogo/templates)

De esta forma, cada parte de la aplicación tiene una función determinada y se mantiene una estructura organizada.



11. Maquetación e interfaz de TechCovers

Una vez preparado y comprobado el funcionamiento del proyecto, se comenzó con la maquetación de la interfaz de TechCovers.

En esta fase se trabaja sobre la apariencia visual de la plataforma, manteniendo la identidad de TechCovers mediante su logotipo, colores, imágenes y estructura de navegación.

La maquetación tiene como idea principal conseguir una interfaz clara, sencilla, estéticamente bonita, que permita al usuario navegar por la plataforma, acceder a las diferentes categorías, consultar los productos disponibles y si le interesa alguno poder hacer una reserva de ese producto/s.

12. Maquetación de páginas

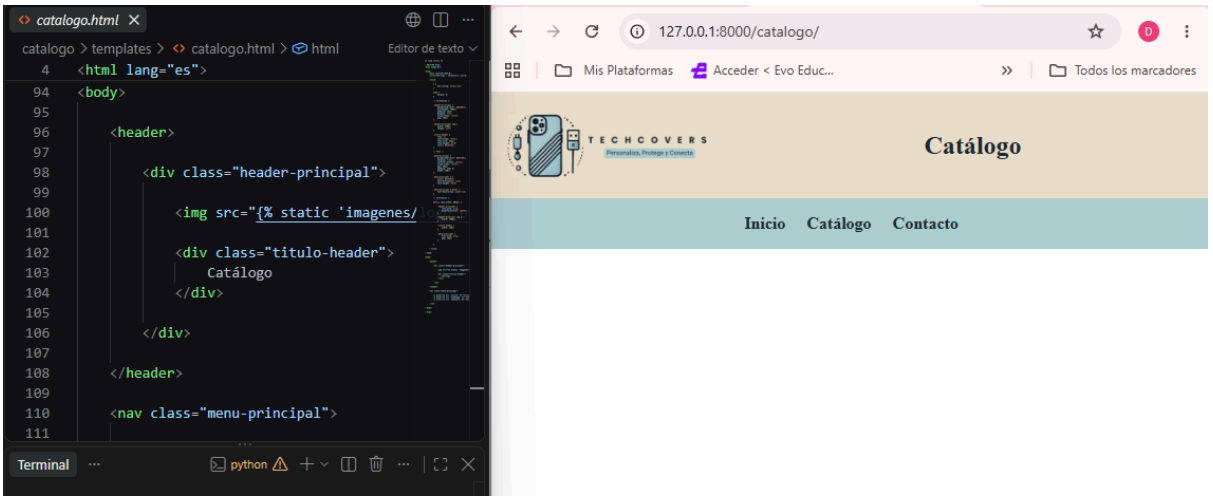
Una vez creadas las distintas páginas de la aplicación, se comenzó a trabajar en su maquetación mediante HTML y CSS, con el objetivo de definir la estructura y la apariencia visual de TechCovers.

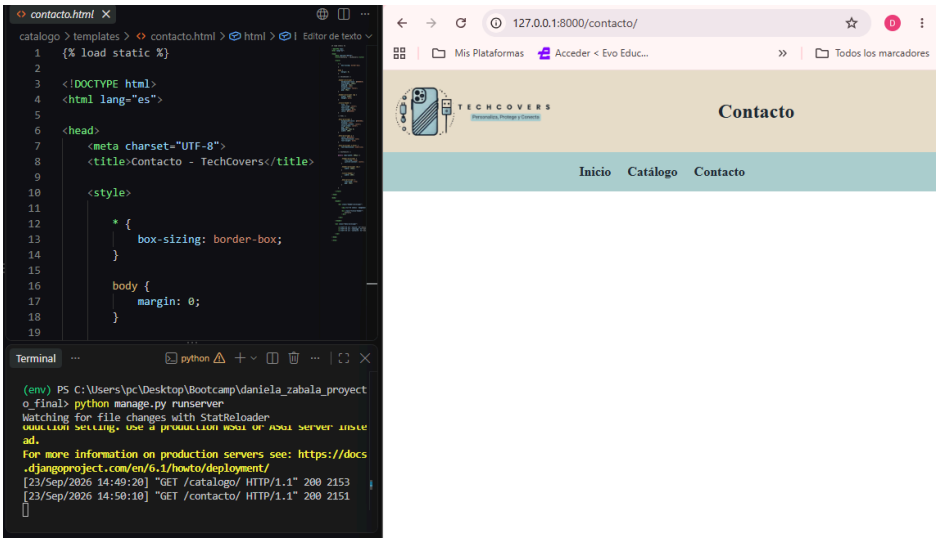
Se fueron definiendo los distintos elementos visuales, como la cabecera, los títulos, los colores, las tipografías, los espacios y la distribución del contenido una vez se tuvo uno visualmente atractivo, se copió en cada página, por lo tanto, cada página tenía su propio código HTML(común) y su configuración de estilos, lo que permitía trabajar en su diseño de forma independiente.

Para desarrollar esta parte del proyecto, se utilizó herramientas de inteligencia artificial, que se consultó para resolver dudas, comprender mejor algunas propiedades de HTML y CSS y explorar posibles soluciones de diseño.

Además, se hizo uso de la documentación oficial de las tecnologías utilizadas para contrastar la información y comprender el funcionamiento de los elementos y estilos incorporados.

- https://developer.mozilla.org/es/docs/Web/HTML
- https://developer.mozilla.org/es/docs/Web/CSS/Guides/Media_queries/Using
- https://developer.mozilla.org/es/docs/Learn_web_development/Getting_started/Your_first_website/Styling_the_content





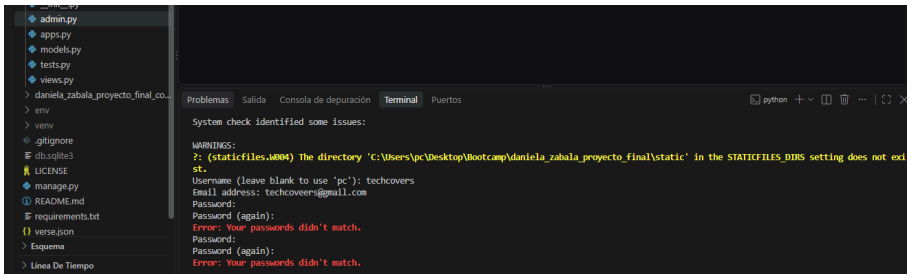
13. Gestión de usuarios

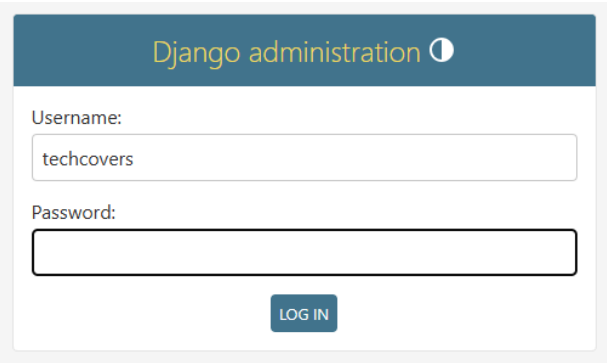
Para comprobar el funcionamiento de la plataforma se crearon diferentes usuarios de prueba mediante el sistema de administración de Django.

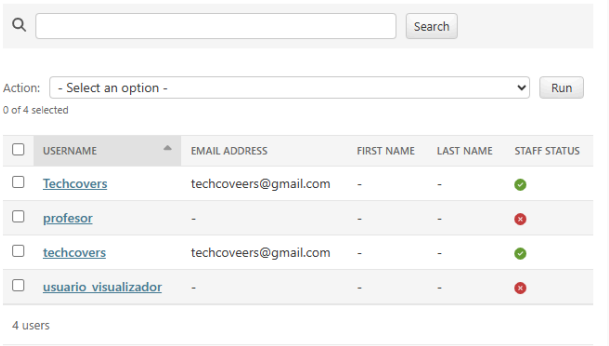
Estos usuarios permiten comprobar el funcionamiento de los diferentes accesos y la gestión de usuarios desde el panel de administración.

También se comprobó que desde Django Admin era posible visualizar y gestionar los usuarios creados.

Posteriormente, se definieron las diferentes vistas que tendría la plataforma: **Inicio, Catálogo y Contacto**.







Usuario	Contraseña
techcovers	prueba1techcovers
profesor	profesor
usuario_visulizador	vertechcovers

Estos usuarios posteriormente se cambiaron ya que la base de datos dejó de ser squlite3 y pasó a ser Neon.

14. Creación de los modelos

Una vez diseñada la estructura de la base de datos, se trasladó dicha estructura al proyecto Django mediante los modelos definidos en models.py.

Se crearon los modelos correspondientes a **Categoría, Producto, Reserva y Producto_reserva**, estableciendo entre ellos las relaciones necesarias.

El modelo Producto contiene la información relacionada con cada accesorio(funda, cable, protector de pantalla, protector de cámara, baterías con cable…) mientras que Categoría permite organizar estos productos.(Sonido, Carga, Audio, Protección ..)

Por otra parte, Reserva permite registrar las reservas realizadas y Producto_reserva relaciona cada reserva con los productos incluidos en ella y con la cantidad correspondiente.

```python
class Categoria(models.Model):
    nombre = models.CharField(max_length=100)
    descripcion = models.TextField()

    def __str__(self):
        return self.nombre


# un product de tech covers tiene nombre, descripcion, precio y stock
#Relacion catgoria producto 1:N

class Producto(models.Model):
    nombre = models.CharField(max_length=200)
    descripcion = models.TextField()
    precio = models.DecimalField(max_digits=10, decimal_places=2)
    stock = models.IntegerField()
    categoria = models.ForeignKey(
        Categoria,
        on_delete=models.CASCADE,
        related_name='productos'
    )

    def __str__(self):
        return self.nombre
```

Después de crear estos modelos, se realizó su conexión con admin.py, permitiendo gestionar estas entidades desde el panel de administración de Django.

catalogo > admin.py

```python
#from django.contrib import admin

from django.contrib import admin
from .models import Categoria, Producto, Reserva, Producto_reserva


admin.site.register(Categoria)
admin.site.register(Producto)
admin.site.register(Reserva)
admin.site.register(Producto_reserva)
```

15. Cambio de la base de datos a Neon

Después de continuar con el desarrollo de TechCovers y realizar las primeras pruebas de la aplicación en el entorno online, se comprobó que SQLite continuaba funcionando como una base de datos local.

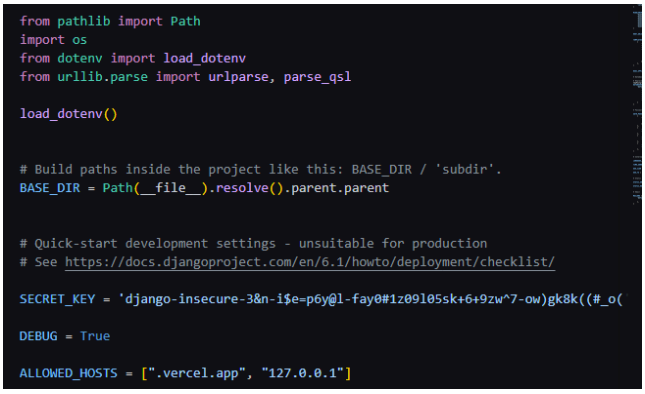
Esto suponía una limitación para el funcionamiento de la aplicación online, ya que era necesario disponer de una base de datos accesible desde el entorno de despliegue.

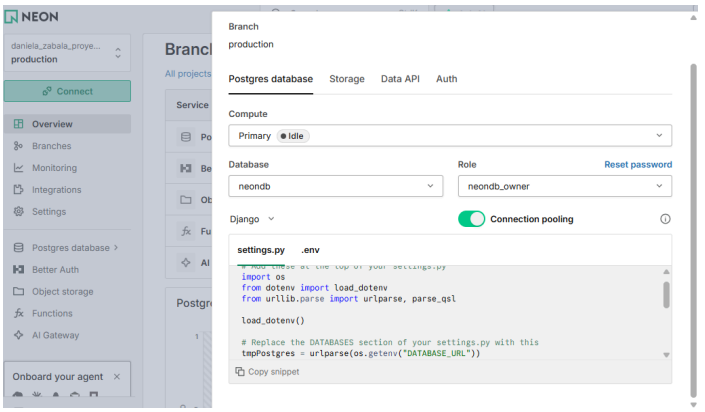
Por este motivo, se decidió cambiar la configuración de la base de datos y realizar la conexión con Neon.

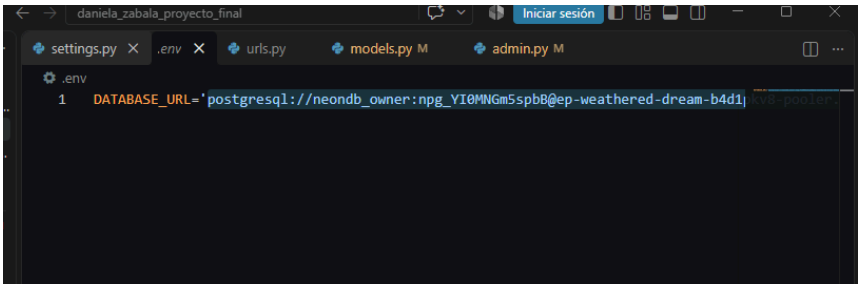
Neon permite utilizar una base de datos PostgreSQL accesible desde el entorno online.

Para realizar el cambio se modificó la configuración de Django y se estableció la conexión mediante la variable de entorno database_url.

A partir de este momento, la aplicación comenzó a utilizar la nueva base de datos online.





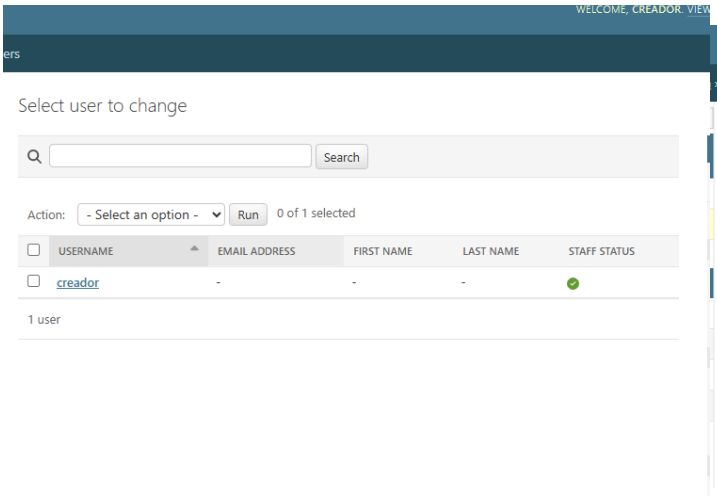


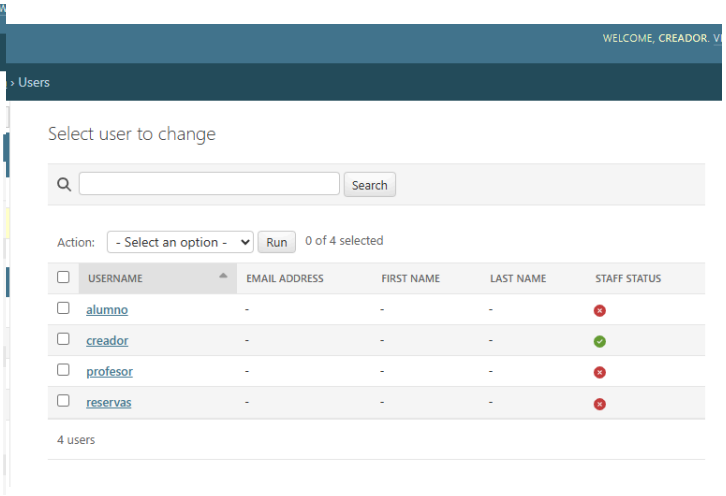
16. Comprobación de la nueva base de datos

Después de realizar la conexión con Neon, se realizaron nuevas comprobaciones para verificar que los datos y usuarios podían gestionarse correctamente utilizando la nueva base de datos.

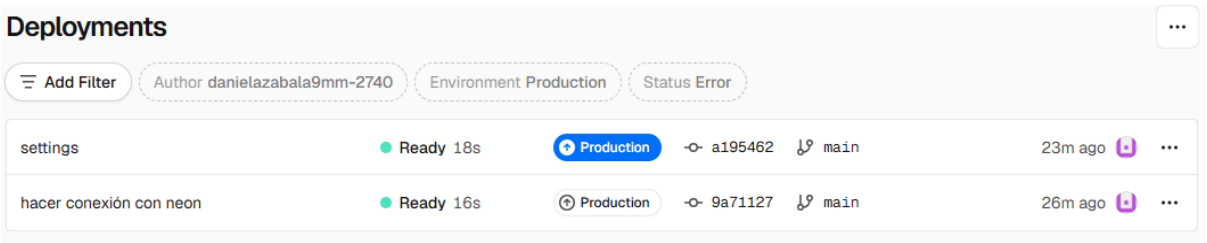
En el panel de administración de Django se comprobó que era posible acceder a la información almacenada (modelos), pero no había usuarios creados, se tuvo que crear nuevos usuarios

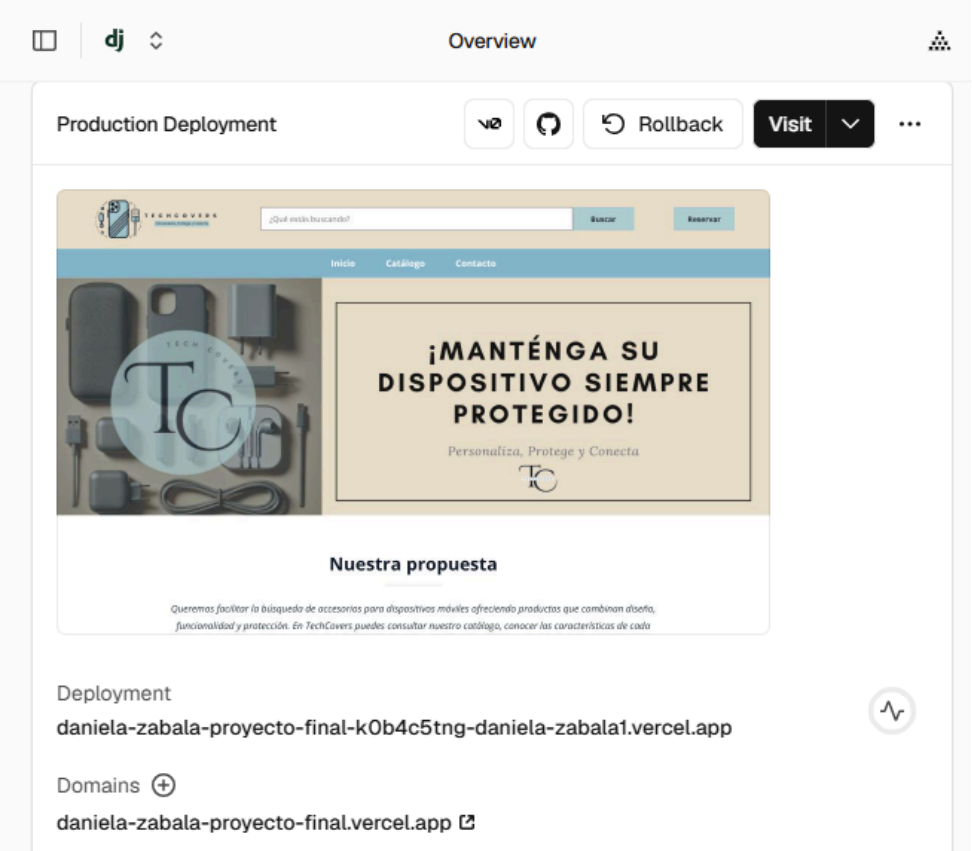
usuario	contraseña
creador —> administrador	123456
profesor	python2026
alumno	python2026
reservas	entrar2026





Probamos que se puede ver en vercel, el panel admin y nos deja entrar.





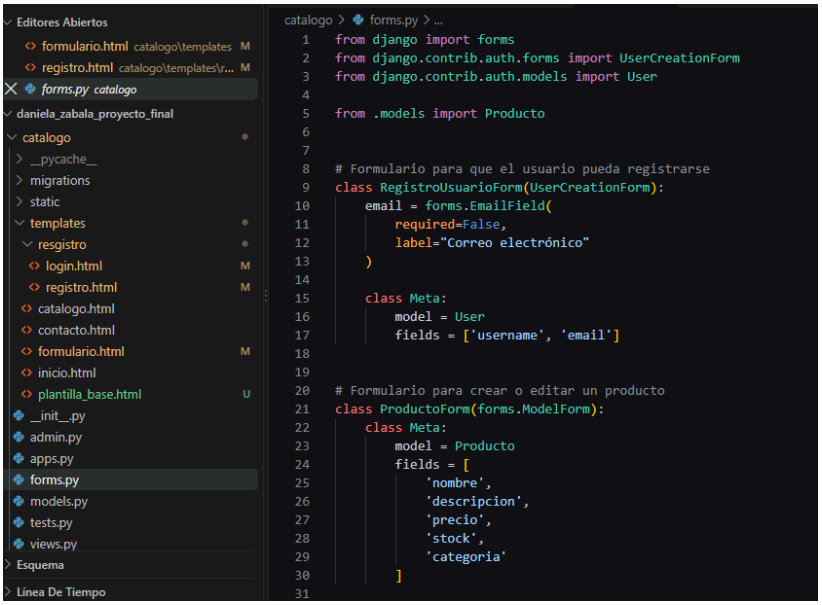
https://daniela-zabala-proyecto-final.vercel.app/

17. Desarrollo de Formularios y Vistas

Después de tener los modelos creados y conectados con Django, seguimos trabajando con los formularios de la web.

Se creó el archivo forms.py, donde se añadió el formulario para registrar usuarios y el formulario para trabajar con los productos.

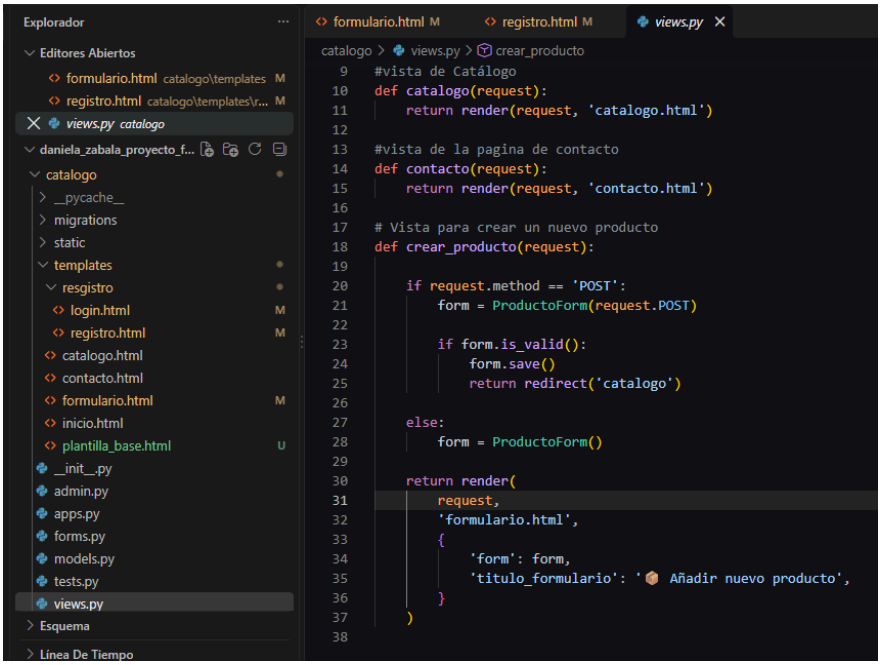
En el caso de los productos, el formulario recoge el nombre, descripción, precio, existencias y categoría.



También se adaptó el formulario que se nos había enseñado en clase para nuestro proyecto. Se cambiaron las referencias que tenía de libros por productos de TechCovers y se adaptaron a nuestro diseño.

Para poder utilizar este formulario se creó una nueva vista en views.py llamada crear_producto.

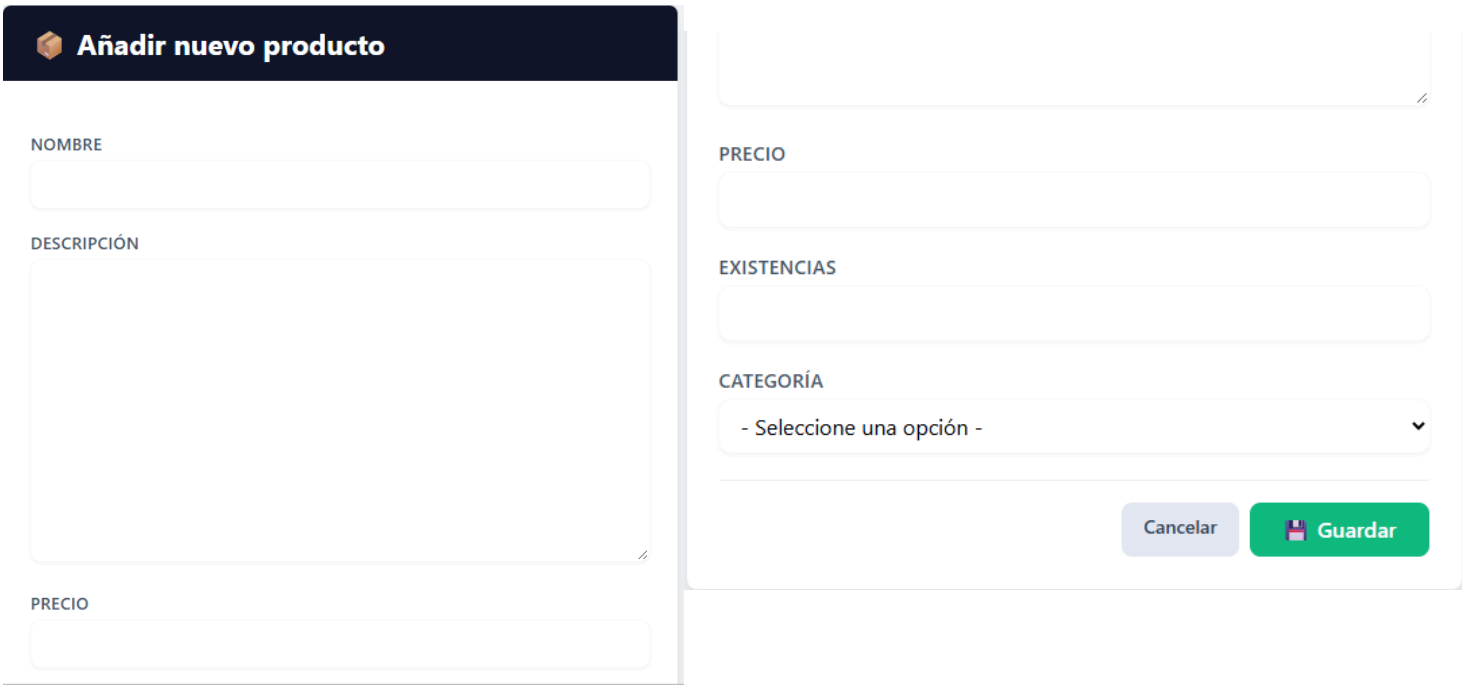
Esta vista se encarga de mostrar el formulario y, cuando se envía, comprobar los datos y guardar el producto.



También se añadió una nueva ruta en urls.py :



Al probarla desde el navegador se pudo comprobar que el formulario se mostraba correctamente.



18.Comprobación de las categorías

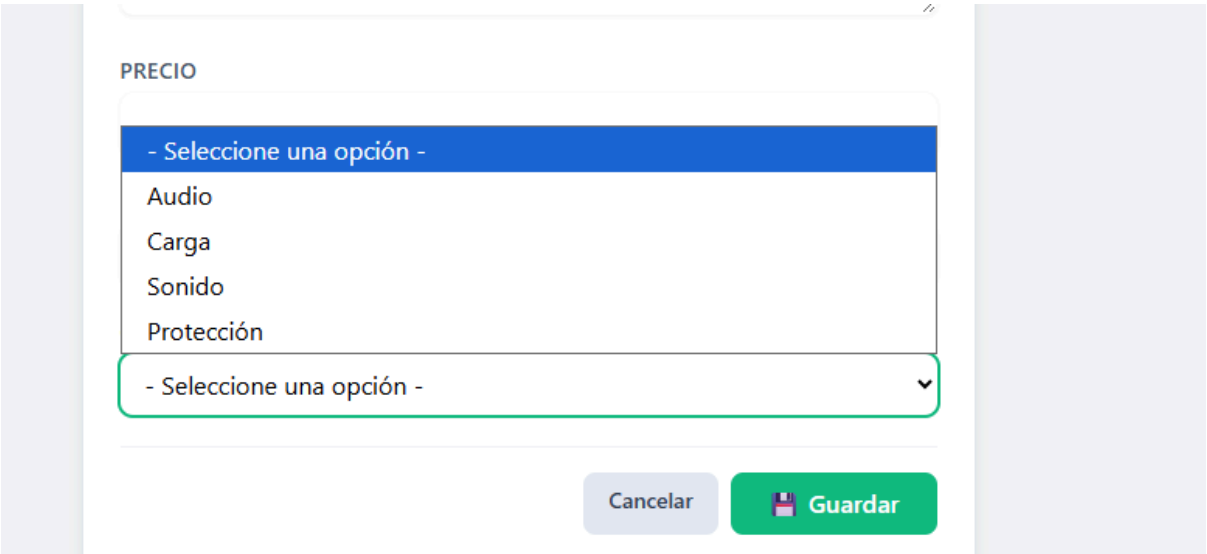
Después se inició el panel de administración de Django para comprobar las categorías que se habían creado.

Se creó y se compró las siguientes categorías:

- Audio
- Carga
- Sonido
- Protección

Al volver al formulario de productos se demostró que estas categorías aparecían automáticamente en el desplegable de categoría.

Esto permitió comprobar que la relación entre Producto y Categoría estaba funcionando correctamente.



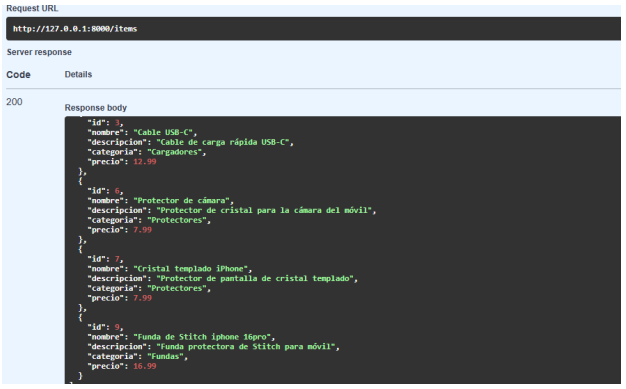
19. Catálogo de Productos

Una vez desarrollada la estructura inicial de la aplicación, se modificó la vista correspondiente al catálogo para conectarla con la base de datos. Para ello, se implementó la consulta de los productos almacenados en la tabla correspondiente al modelo Producto, de manera que la información recuperada pudiera mostrarse dinámicamente en la plantilla catalogo.html.

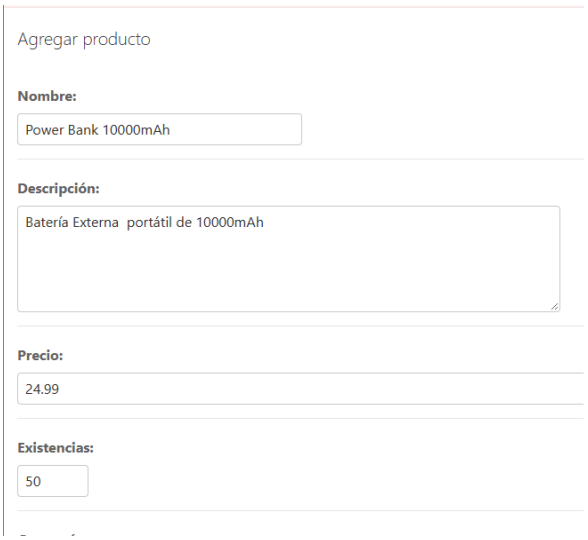
Posteriormente, se adaptó la plantilla para representar los productos obtenidos de la base de datos. También se incorporó una condición para contemplar el caso de que no hubiera productos registrados, mostrando un mensaje informativo al usuario en lugar de dejar la página vacía.

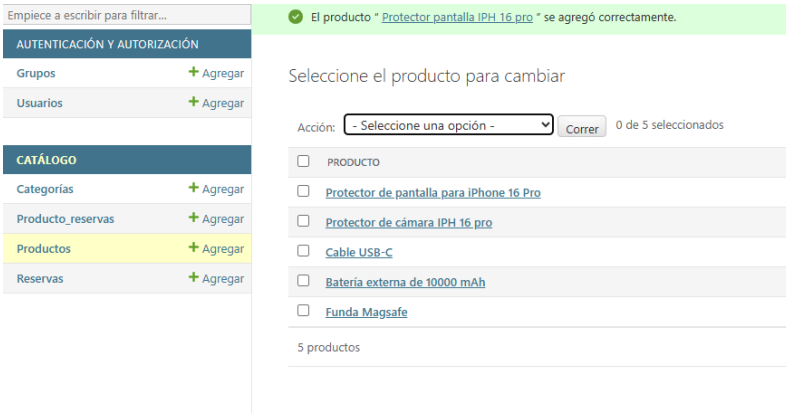


Para comprobar el funcionamiento del catálogo, se añadieron los productos utilizados en la PEC 4, adaptándolos a las categorías definidas para TechCovers. De esta forma, se pudo verificar que los productos se recuperaban y mostraban correctamente, y que la información presentada dependía de los datos almacenados en la base de datos.






Agregar producto
Nombre:
Power Bank 10000mAh
Descripción:
Batería Externa portátil de 10000mAh
Precio:
24.99
Existencias:
50


Empiece a escribir para filtrar...
AUTENTICACIÓN Y AUTORIZACIÓN
Grupos + Agregar
Usuarios + Agregar
CATÁLOGO
Categorías + Agregar
Producto_reservas + Agregar
Productos + Agregar
Reservas + Agregar
El producto " Protector pantalla IPH 16 pro " se agregó correctamente.
Seleccione el producto para cambiar
Acción: - Seleccione una opción - Correr 0 de 5 seleccionados
PRODUCTO
Protector de pantalla para iPhone 16 Pro
Protector de cámara IPH 16 pro
Cable USB-C
Batería externa de 10000 mAh
Funda Magsafe
5 productos

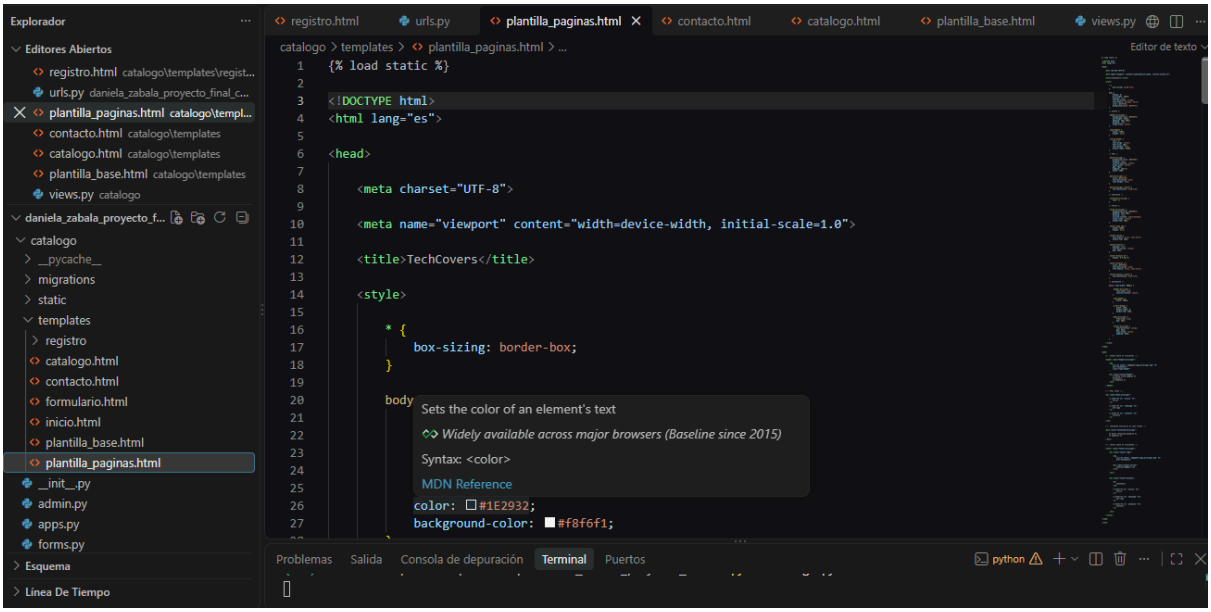

TECHCOVERS
Catálogo
Inicio Catálogo Contacto
Productos disponibles
Consulta nuestros accesorios para dispositivos móviles.
Protección
Funda Magsafe
Funda transparente compatible con Magsafe
19.99 €
Existencias: 50
Carga
Power Bank 10000mAh

20. Plantillas comunes

Durante la revisión del proyecto, se detectó que a pesar de que en un principio se había copiado y pegado el mismo estilo, cada página tenía su propia maquetación para la cabecera y el pie de página, lo que generaba diferencias visuales entre las distintas secciones de la web y dificultaba el mantenimiento del código

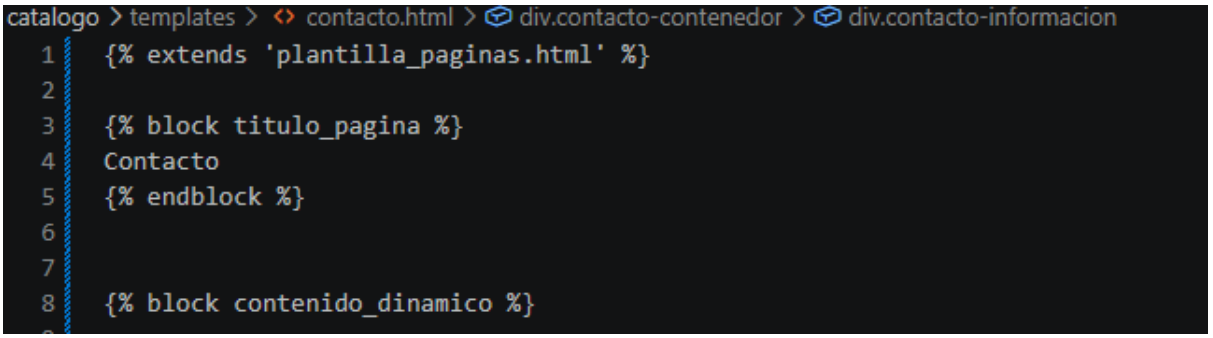


Para solucionar este problema, se creó una plantilla común denominada plantilla_paginas.html, destinada a centralizar los elementos compartidos por todas las páginas de TechCovers, como la cabecera, el menú de navegación y el pie de página.



A partir de esta modificación, las nuevas plantillas pudieron heredar la estructura común mediante la instrucción {% extends 'plantilla_paginas.html' %}. De esta forma, cada página conserva su contenido específico, pero utiliza los mismos elementos de navegación y diseño general.

Como resultado, se consiguió unificar la apariencia de las distintas secciones de TechCovers, mejorar la coherencia visual de la aplicación y facilitar futuras modificaciones, ya que los cambios en la cabecera o el pie de página pueden realizarse desde un único archivo.









21. Registro e inicio de sesión

Una vez que el catálogo y los productos ya funcionaban correctamente, el siguiente paso fue permitir que los usuarios pudieran registrarse e iniciar sesión en TechCovers.

El objetivo era diferenciar entre los usuarios que habían iniciado sesión y los que todavía no lo habían hecho, ya que más adelante las reservas estarían asociadas a la cuenta de cada usuario.

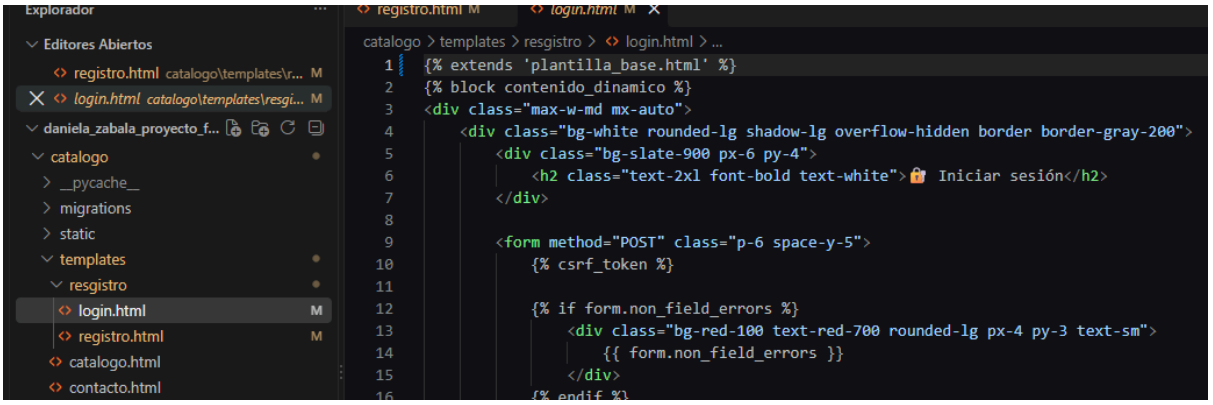
Para ello, se utilizaron las herramientas de autenticación que proporciona Django, que facilitaron la incorporación del registro y el inicio de sesión en la aplicación.

22.Creación de las plantilla de Registro e inicio de sesión

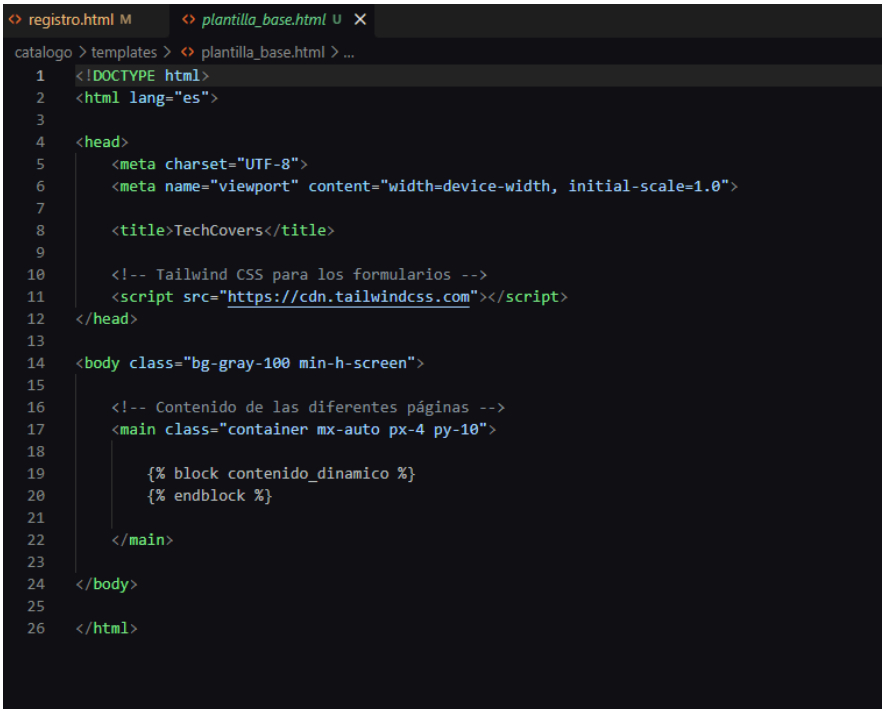
Dentro de la carpeta templates se creó una carpeta llamada registro, donde se guardaron las dos páginas relacionadas con los usuarios:

- login.html, utilizada para iniciar sesión.
- registro.html, utilizada para crear una cuenta nueva.

Estas plantillas se prepararon utilizando los formularios de Django y permiten mostrar los campos necesarios para que el usuario pueda iniciar sesión o registrarse en la aplicación.



También se creó una plantilla común llamada plantilla_base.html, que se utilizó como estructura para algunas de las páginas. Al igual que con plantilla_paginas.html, el objetivo era reutilizar una misma estructura y colocar dentro el contenido específico de cada página, evitando repetir código y facilitando su organización.



23. Conexión del registro con Django

Una vez creadas las plantillas HTML, se conectó el formulario de registro con Django.

Para ello se utilizó el formulario RegistroUsuarioForm definido anteriormente en forms.py.

Después se creó en views.py. la función:

```python

# Vista para registrar un nuevo usuario
def registro(request):

    if request.method == 'POST':
        form = RegistroUsuarioForm(request.POST)

        if form.is_valid():
            usuario = form.save()

            login(request, usuario)

            return redirect('inicio')

    else:
        form = RegistroUsuarioForm()

    return render(
        request,
        'registro/registro.html',
        {
            'form': form
        }
    )

```

24. Configuración de la ruta de registro

una vez hecho esto, hay que crear la ruta del registro en urls.py

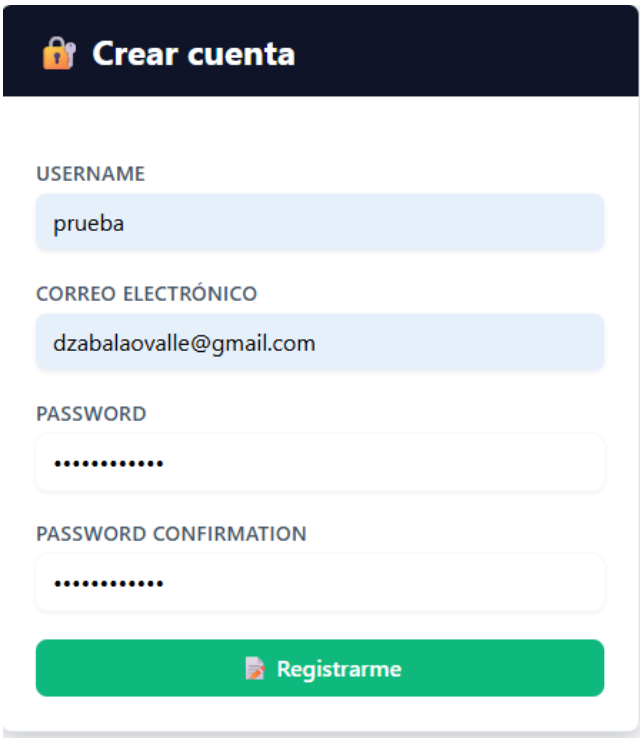
```python
    #registro de usuarios
    path(
        'accounts/registro/',
        views.registro,
        name='registro'
    ),
```

25. Comprobación del registro

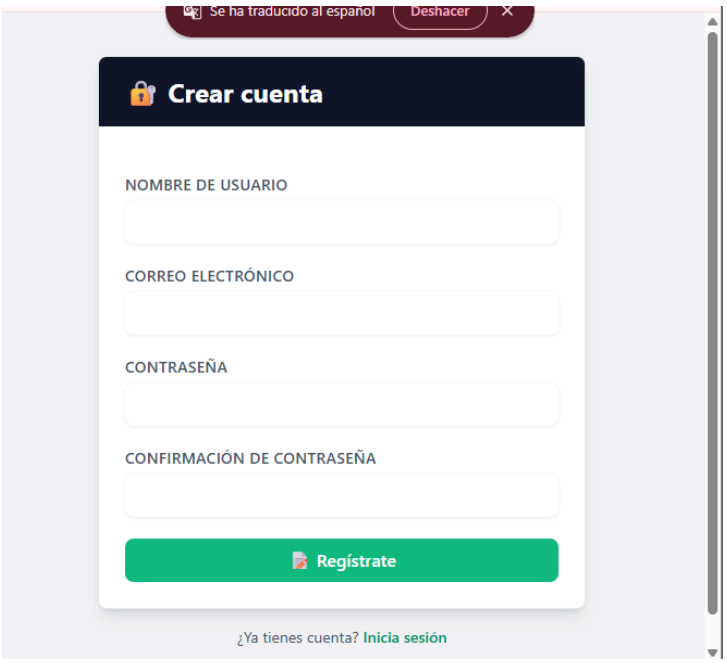
Una vez conectada la plantilla con Django, se realizó una prueba desde el navegador para comprobar que el formulario funcionaba correctamente.

Se comprobó que el usuario podía introducir sus datos, enviar el formulario y crear una cuenta.

También se comprobó posteriormente desde el panel de administración de Django que el usuario había sido creado correctamente.

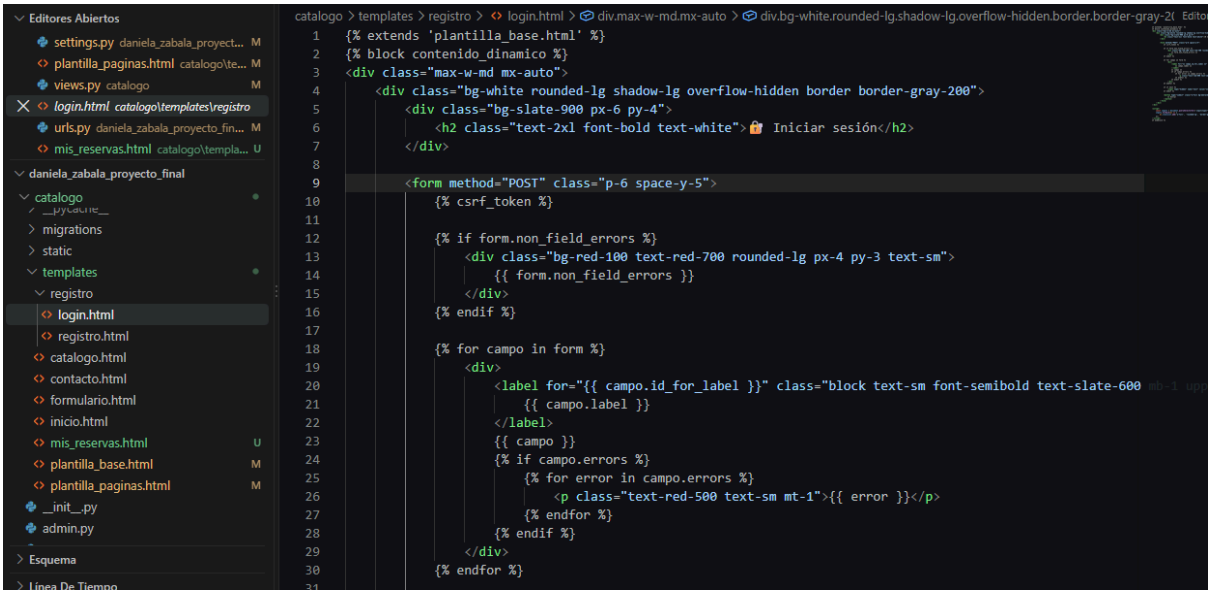


Usuario	Contraseña
prueba	123456prueba



26. Configuración Inicio de sesión y cierre

Después de comprobar el registro, se conectó la página de inicio de sesión con el sistema de autenticación de Django.



Para gestionar el inicio de sesión y de cierre de sesión se utiliza el siste,a de autenticación de Django, de esta manera Django procesa los credenciales introducidos y si el usuario pulsa el botón de salir, la sesión activa finaliza.

```python
#inicio de sesion
path(
    'accounts/login/',
    auth_views.LoginView.as_view(
        template_name='registro/login.html'
    ),
    name='login'
),

#cierre de sesion
path(
    'accounts/logout/',
    auth_views.LogoutView.as_view(),
    name='logout'
),
```

27. Redirecciones

También se configuraron en settings.py. las rutas de redirección para controlar a qué página accede el usuario después de iniciar o cerrar sesión.

De esta forma, cuando inicia sesión correctamente, es redirigido al catálogo de productos, mientras que, al cerrar sesión, vuelve a la página de inicio.

Con esta configuración, la navegación resulta más sencilla y el comportamiento de la aplicación es más coherente con las acciones que realiza cada usuario.

```python
}

LOGIN_REDIRECT_URL = '/catalogo/'
LOGOUT_REDIRECT_URL = '/'
```

28. Menú adaptado al estado del usuario

Una vez que el sistema de autenticación estaba funcionando, se modificó la plantilla común para que el menú cambiará dependiendo de si el usuario había iniciado sesión

Si el usuario está autenticado, se muestra:

- **Su nombre de usuario**
- **Mis reservas**
- **Salir.**

Si no está autenticado, se muestran:

- **Registrarse**
- **Iniciar sesión**

De esta forma, la interfaz se adapta al estado de la sesión.





29.Creación de la sección "Mis reservas"

Una vez que el registro y el inicio de sesión estuvieron funcionando, se continuó con la gestión de las reservas.

Siguiendo la estructura definida para el proyecto, se creó una nueva vista denominada: mis_reservas

Esta vista comprueba primero si el usuario está autenticado.

Si el usuario no ha iniciado sesión, se le redirige a la página de Login.

Si está autenticado, se consultan las reservas asociadas a ese usuario mediante el ORM de Django:

```python
#Vista para consultar las reservas del usuario
def mis_reservas(request):

    if request.user.is_authenticated:

        reservas = Reserva.objects.filter(
            usuario=request.user
        )

        return render(
            request,
            'mis_reservas.html',
            {
                'reservas': reservas
            }
        )

    return redirect('login')
```

30. Creación de las páginas de error 404 y 500

Durante las pruebas de navegación de la web se decidió personalizar la página que se muestra cuando un usuario intenta acceder a una dirección que no existe.

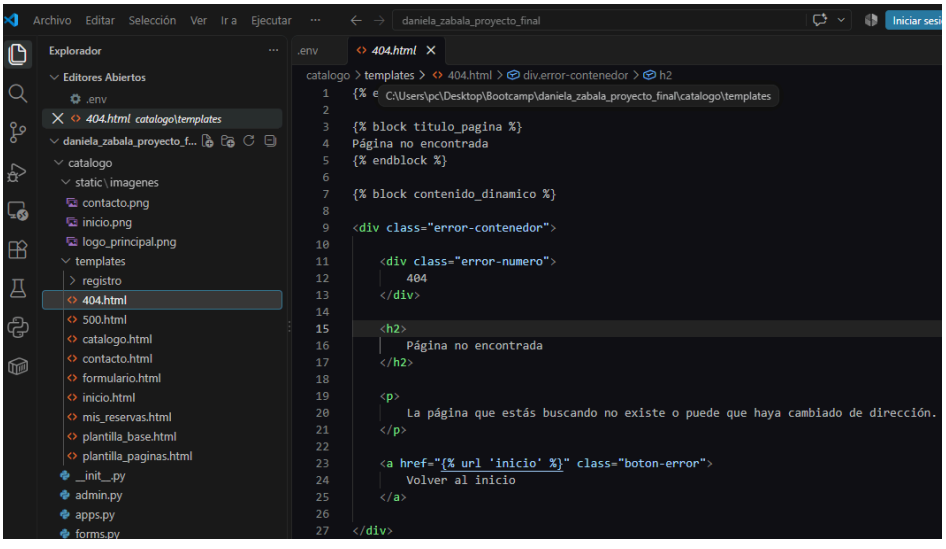
Para ello, se creó una plantilla denominada 404.html dentro de la carpeta templates de la aplicación catálogo.

La plantilla utiliza plantilla_paginas.html como base, manteniendo así el mismo encabezado, menú de navegación y pie de página que el resto de las páginas de TechCovers.

En la página se muestra el código de error **404**, acompañado de un mensaje indicando que la página solicitada no existe o puede haber cambiado de dirección.

También se añadió un botón que permite volver directamente a la página de inicio:





31. Control de permisos para la creación de productos

Una vez comprobado el funcionamiento de las páginas de error, se continuó con la gestión de permisos de los usuarios.

Se estableció que la creación de nuevos productos debe estar limitada a los usuarios que dispongan de permisos de administración.

Para ello, se utilizó el sistema de permisos integrado de Django mediante el decorador permission_required.

En la vista crear_producto se añadió el permiso:

```
# Vista para crear un nuevo producto
@permission_required(
    'catalogo.add_producto',
    raise_exception=True
)  #limitar al adm para que solo ese usuario pueda crear prod

def crear_producto(request):
    if request.method == 'POST':
        form = ProductoForm(request.POST)
```

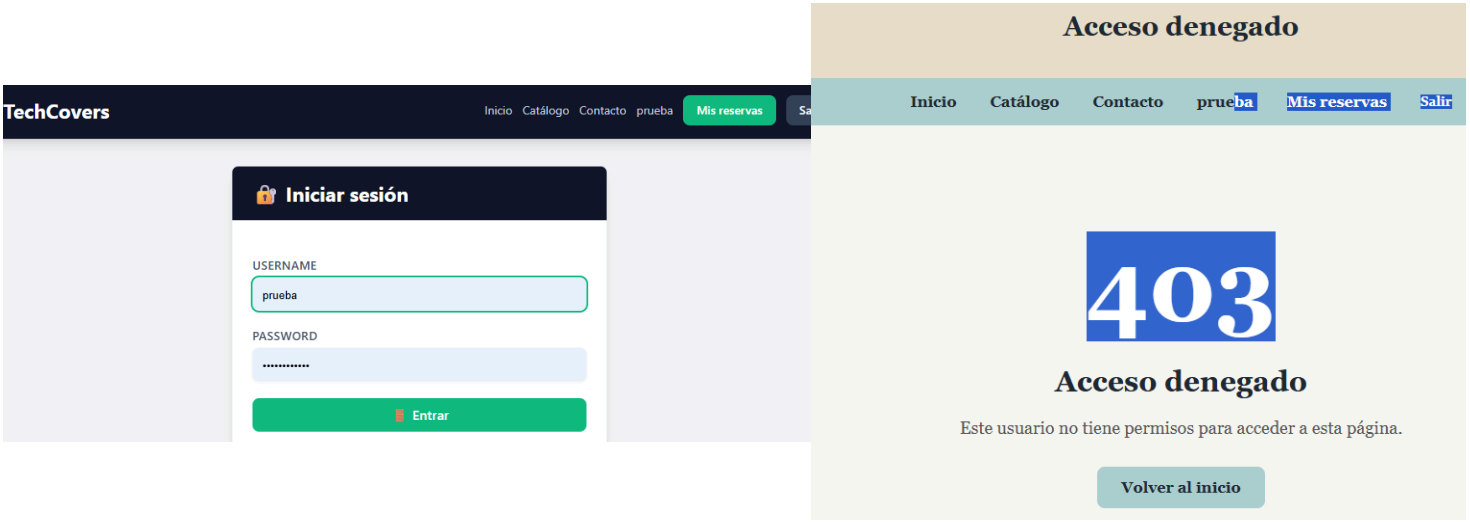
Este permiso permite controlar el acceso a la vista encargada de crear nuevos productos.

De esta forma, los usuarios que no dispongan del permiso necesario no pueden acceder al formulario de creación aunque intenten introducir directamente la dirección de la página en el navegador.

Para comprobar este funcionamiento se realizó una prueba utilizando el usuario prueba, que no dispone de permisos para añadir productos. Al intentar acceder a la página de creación, Django mostró un error **403 – Acceso denegado**.

Para mejorar la presentación de este error, se creó una nueva plantilla denominada 403.html.

Esta plantilla utiliza plantilla_paginas.html mediante herencia, por lo que mantiene la estructura visual común de TechCovers, incluyendo el encabezado, menú y pie de página.



32. Gestión y modificación de productos

Después de comprobar que los usuarios sin permisos recibían el error 403 – Acceso denegado, se continuó con las opciones de gestión de los productos.

En primer lugar, se añadió la posibilidad de editar los productos existentes. De esta forma, el administrador puede seleccionar un producto del catálogo, modificar sus datos y guardar los cambios.

Para ello, se añadió una opción Editar en cada producto del catálogo, visible únicamente para los usuarios que tienen permisos para realizar cambios.

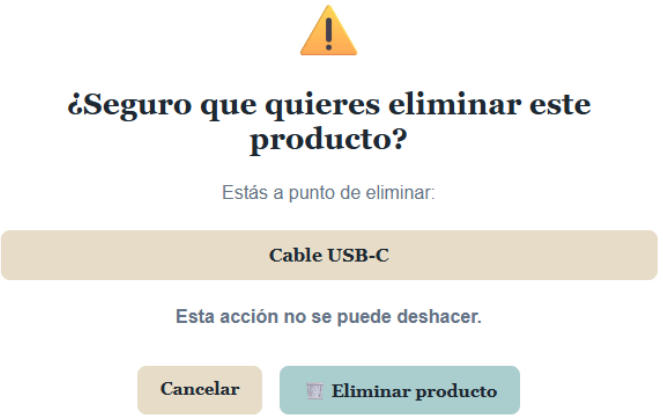
También se añadió la posibilidad de eliminar productos. Para evitar eliminar un producto por error, antes de realizar la eliminación se muestra una pantalla de confirmación.

En esta pantalla aparece el nombre del producto y dos opciones: Cancelar o Eliminar producto.

Al confirmar la eliminación, el producto desaparece del catálogo.

Otro usuario diferente al administrador no puede hacer estas funciones





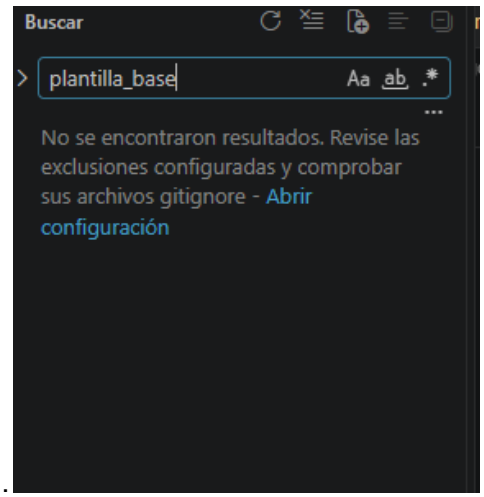


33. Unificación de las plantillas y mejora de la navegación

Después de completar las diferentes páginas de TechCovers, se revisó la estructura de las plantillas para evitar la duplicación de código. Hasta ese momento, se utilizaban dos plantillas comunes:

- plantilla_base.html
- plantilla_paginas.html

Para simplificar la estructura, se unificaron ambas plantillas en un único archivo, plantilla_paginas.html, y se adaptaron las demás páginas para que utilizaran esta nueva estructura. Una vez realizados los cambios, se comprobó que ninguna página siguiera utilizando la plantilla antigua y, finalmente, se eliminó plantilla_base.html.



También se modificó la distribución del menú de navegación. Las opciones principales quedaron centradas en la franja azul, con los siguientes enlaces:

- Inicio
- Catálogo
- Contacto

Por otro lado, la información relacionada con el usuario se situó en la parte superior derecha de la cabecera. Cuando el usuario ha iniciado sesión, aparecen su nombre, la opción Mis reservas y el botón Salir. Cuando no ha iniciado sesión, se muestran las opciones Iniciar sesió y Registrarse.

Con estos cambios se consiguió una interfaz más sencilla, ordenada y uniforme en todas las páginas de TechCovers, además de facilitar el mantenimiento del código al centralizar los elementos comunes en una única plantilla.





34. Gestión de Reservas

La sección Mis reservas estaba creada, pero todavía no llevaba a ninguna página funcional. Por ello, se continuó desarrollando esta parte de TechCovers para permitir que los usuarios pudieran consultar y gestionar sus reservas.

Para ello, se crearon tres plantillas HTML: crear_reserva.html, editar_reserva.html y confirmar_eliminacion_reserva.html.

Las tres heredan de plantilla_paginas.html, manteniendo la estructura visual común de TechCovers y definiendo su contenido dentro del bloque contenido_dinamico.

- **crear_reserva.html:** permite iniciar una reserva desde la ficha del producto. Muestra la información del producto seleccionado —categoría, nombre, descripción, precio y existencias— e incluye un formulario para indicar la cantidad deseada.

- **editar_reserva.html:** permite modificar una reserva existente. Muestra su número identificativo, los datos del producto y el stock disponible, e incluye un formulario para introducir la nueva cantidad. También ofrece opciones para cancelar o guardar los cambios y muestra los posibles mensajes de error.

- **confirmar_eliminacion_reserva.html:** muestra una pantalla de confirmación antes de eliminar una reserva. Permite revisar la información correspondiente y elegir entre cancelar la operación o confirmar la eliminación, evitando así borrar una reserva por error.

Además de crear estas plantillas, se desarrolló la lógica correspondiente en las vistas de Django para gestionar las operaciones. Se incorporaron las comprobaciones necesarias para validar las cantidades solicitadas y mantener la coherencia entre las reservas y las existencias disponibles. De esta forma, al modificar o eliminar una reserva, se tiene en cuenta el stock para actualizarlo correctamente cuando corresponde.

Con estas modificaciones, la sección Mis reservas pasó a formar parte de las funcionalidades de TechCovers, permitiendo gestionar las reservas desde la propia web.



35. Pruebas Automáticas con Django.

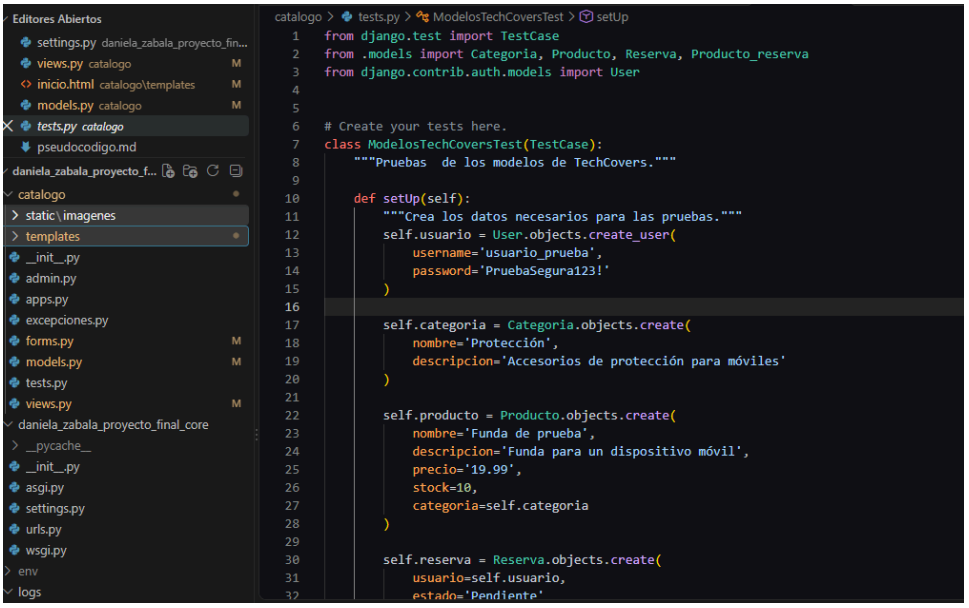
Para comprobar parte del funcionamiento interno de TechCovers se incorporaron pruebas automáticas en tests.py. Django ofrece herramientas para preparar datos, ejecutar comprobaciones y verificar si los resultados coinciden con los esperados.

La documentación oficial consultada es:

- https://docs.djangoproject.com/en/6.0/topics/testing/tools/.

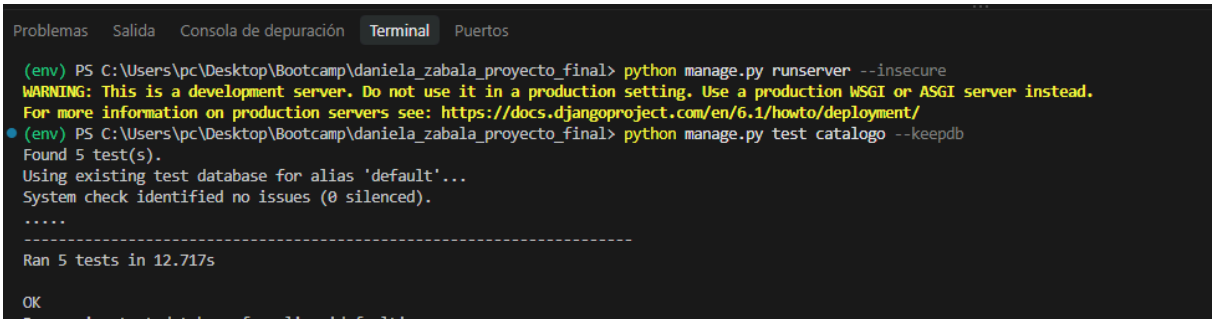
Las cinco pruebas incluidas comprueban los siguientes aspectos:

- test_nombre_categoria: verifica que la representación textual de una categoría devuelve su nombre.
- test_nombre_producto: comprueba que la representación textual de un producto devuelve su nombre.
- test_reserva_asociada_usuario: comprueba que la reserva pertenece al usuario esperado.
- test_producto_asociado_reserva: verifica la cantidad reservada y la relación con el producto.
- test_relacion_categoria_producto: comprueba la relación entre el producto y su categoría, incluida la relación inversa.



Las pruebas se ejecutaron desde la terminal con *python manage.py test catalogo --keepdb.* En la ejecución comprobada, las cinco pruebas finalizaron correctamente.

Este resultado valida las relaciones y representaciones cubiertas por estos tests.



36. Docstrings

Durante la revisión del código se añadieron docstrings para explicar la finalidad de módulos, clases y métodos. Son cadenas de documentación situadas al comienzo de un módulo, una clase o una función/método, y permiten entender su cometido sin deducirlo únicamente del código.

En TechCovers se utilizan, por ejemplo, docstrings en las vistas de views.py para indicar qué página o acción gestionan; en los formularios de forms.py para explicar su función, en los modelos models.py. para indicar los modelos y en la clase ValidadorStock para documentar la validación del stock disponible.

.

```python
from django.db import models
#mi tabla de reserva este conectada con con usuarios
from django.contrib.auth.models import User


class Categoria(models.Model):
    """Representa una categoría de productos."""
    nombre = models.CharField(max_length=100)
    descripcion = models.TextField()

    def __str__(self):
        return self.nombre


# un product de tech covers tiene nombre, descripcion, precio y stock
#Relacion catgoria producto 1:N

class Producto(models.Model):
    """Representa un accesorio del catálogo."""
    nombre = models.CharField(max_length=200)
    descripcion = models.TextField()
    precio = models.DecimalField(max_digits=10, decimal_places=2)
    stock = models.IntegerField()
    categoria = models.ForeignKey(
        Categoria,
        on_delete=models.CASCADE,
        related_name='productos'
    )
```

ValidadorStock encapsula el stock en un atributo privado y dispone del método validar, que comprueba si la cantidad solicitada supera las existencias disponibles. Cuando no hay stock suficiente, se utiliza la excepción personalizada StockInsuficienteError.

```python
class ValidadorStock:
    """Valida el stock disponible para las reservas."""

    def __init__(self, stock):
        """Guarda el stock en un atributo privado."""
        self.__stock = stock

    def validar(self, cantidad):
        """Comprueba que la cantidad no supere el stock."""
        if cantidad > self.__stock:
            raise StockInsuficienteError(
                "No hay suficientes existencias disponibles."
            )
```

37. Rediseño página de Inicio.

La página de inicio ha cambiado, con respecto a la recogida al principio de la memoria, la página de inicio era el único HTML independiente, que no utilizaba plantillas.

Se ha cambiado, para que todas las páginas respeten el diseño de Techcovers y a la vez para que el usuario pueda tener una navegación más sencilla por la web.



38. Logs.

Se incorporó un sistema de *logging* para registrar eventos relevantes durante la ejecución de TechCovers. Para ello, se utilizó el método de registro de eventos que ya se había trabajado en la PEC 2, adaptándolo a las necesidades del proyecto.

Este sistema permite consultar lo que ocurre durante la ejecución de la aplicación y facilita la detección de posibles incidencias.

En views.py se obtiene un logger asociado al módulo mediante logging.getLogger(**__name___**)

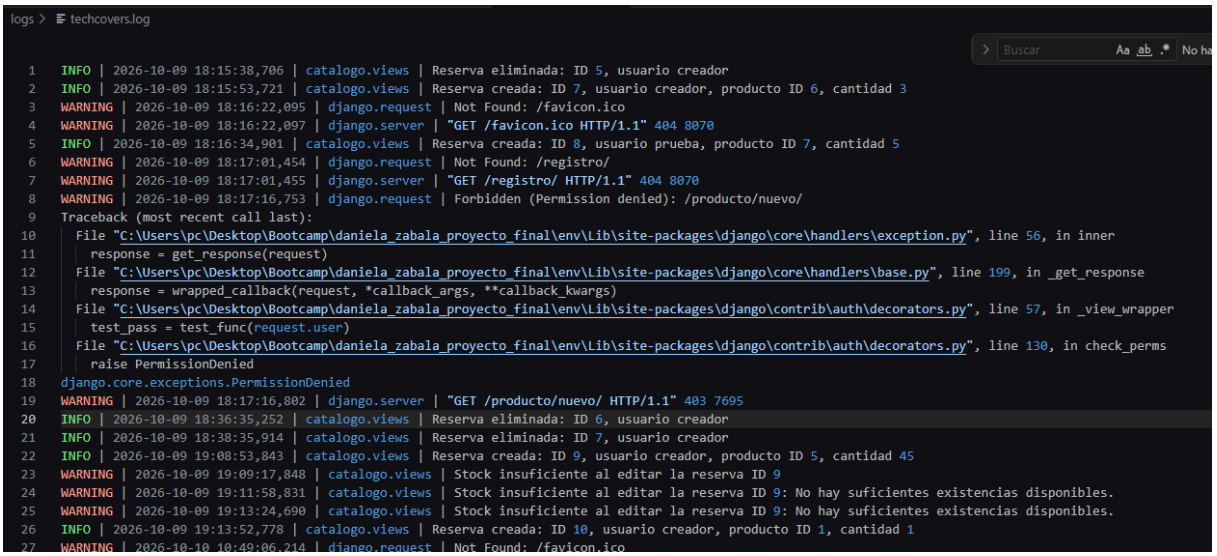
En el entorno local, los mensajes se guardan en el archivo logs/techcovers.log, lo que permite consultarlos posteriormente y revisar los eventos registrados.

Además, se adaptó la configuración para el despliegue en Vercel. En este entorno se evita depender de un archivo de registro local y se utiliza la salida de consola, ya que el sistema de archivos puede tener restricciones o no ser persistente. De esta manera, se mantiene el registro de eventos tanto durante el desarrollo local como en el entorno online.

```python
#Configuración del logger
logger = logging.getLogger(__name__)


class ValidadorStock:
    """Valida el stock disponible para las reservas."""

    def __init__(self, stock):
        """Guarda el stock en un atributo privado."""
        self.__stock = stock
```



```
logs > techcovers.log

1   INFO | 2026-10-09 18:15:38,706 | catalogo.views | Reserva eliminada: ID 5, usuario creador
2   INFO | 2026-10-09 18:15:53,721 | catalogo.views | Reserva creada: ID 7, usuario creador, producto ID 6, cantidad 3
3   WARNING | 2026-10-09 18:16:22,095 | django.request | Not Found: /favicon.ico
4   WARNING | 2026-10-09 18:16:22,097 | django.server | "GET /favicon.ico HTTP/1.1" 404 8070
5   INFO | 2026-10-09 18:16:34,901 | catalogo.views | Reserva creada: ID 8, usuario prueba, producto ID 7, cantidad 5
6   WARNING | 2026-10-09 18:17:01,454 | django.request | Not Found: /registro/
7   WARNING | 2026-10-09 18:17:01,455 | django.server | "GET /registro/ HTTP/1.1" 404 8070
8   WARNING | 2026-10-09 18:17:16,753 | django.request | Forbidden (Permission denied): /producto/nuevo/
9   Traceback (most recent call last):
10    File "C:\Users\pc\Desktop\Bootcamp\daniela_zabala_proyecto_final\env\Lib\site-packages\django\core\handlers\exception.py", line 56, in inner
11      response = get_response(request)
12    File "C:\Users\pc\Desktop\Bootcamp\daniela_zabala_proyecto_final\env\Lib\site-packages\django\core\handlers\base.py", line 199, in _get_response
13      response = wrapped_callback(request, *callback_args, **callback_kwargs)
14    File "C:\Users\pc\Desktop\Bootcamp\daniela_zabala_proyecto_final\env\Lib\site-packages\django\contrib\auth\decorators.py", line 57, in _view_wrapper
15      test_pass = test_func(request.user)
16    File "C:\Users\pc\Desktop\Bootcamp\daniela_zabala_proyecto_final\env\Lib\site-packages\django\contrib\auth\decorators.py", line 130, in check_perms
17      raise PermissionDenied
18  django.core.exceptions.PermissionDenied
19  WARNING | 2026-10-09 18:17:16,802 | django.server | "GET /producto/nuevo/ HTTP/1.1" 403 7695
20  INFO | 2026-10-09 18:36:35,252 | catalogo.views | Reserva eliminada: ID 6, usuario creador
21  INFO | 2026-10-09 18:38:35,914 | catalogo.views | Reserva eliminada: ID 7, usuario creador
22  INFO | 2026-10-09 19:08:53,843 | catalogo.views | Reserva creada: ID 9, usuario creador, producto ID 5, cantidad 45
23  WARNING | 2026-10-09 19:09:17,848 | catalogo.views | Stock insuficiente al editar la reserva ID 9
24  WARNING | 2026-10-09 19:11:58,831 | catalogo.views | Stock insuficiente al editar la reserva ID 9: No hay suficientes existencias disponibles.
25  WARNING | 2026-10-09 19:13:24,690 | catalogo.views | Stock insuficiente al editar la reserva ID 9: No hay suficientes existencias disponibles.
26  INFO | 2026-10-09 19:13:52,778 | catalogo.views | Reserva creada: ID 10, usuario creador, producto ID 1, cantidad 1
27  WARNING | 2026-10-10 10:49:06,214 | django.request | Not Found: /favicon.ico
```

39. Pseudocódigo

Como apoyo a la implementación se preparó pseudocodigo.md, que describe de forma simplificada los principales procesos de TechCovers sin utilizar la sintaxis específica de Python o Django.

El flujo descrito incluye:

- Mostrar la página inicial y permitir consultar el catálogo.
- Crear una reserva, solicitar la cantidad y comprobar que existe stock suficiente.
- Actualizar las existencias cuando se crea o modifica una reserva.
- Mostrar un mensaje de error si la cantidad solicitada supera el stock disponible.
- Eliminar una reserva y devolver las unidades correspondientes al stock.
- Permitir al administrador crear, modificar y eliminar productos.

El pseudocódigo ofrece una visión general del comportamiento esperado.

La implementación real se encuentra en las vistas, los modelos y las plantillas del proyecto.

pseudocodigo.md

```
// --- Inicio ---

MOSTRAR "--- TechCovers ---"

// --- Cliente ---

// --- Consultar catálogo ---

MOSTRAR catálogo de productos

// --- Crear reserva ---

SI el usuario quiere reservar ENTONCES

    PEDIR cantidad

    COMPROBAR stock disponible

    SI hay stock suficiente ENTONCES

        CREAR reserva
        ACTUALIZAR stock

        MOSTRAR "Reserva creada correctamente"

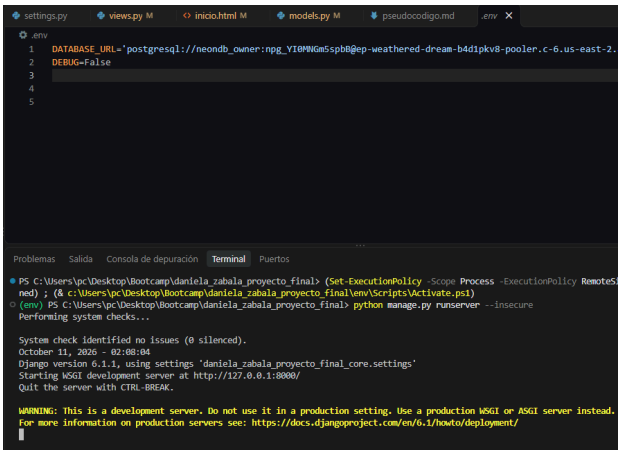
    SINO

        MOSTRAR "No hay stock suficiente"

    FIN SI
```

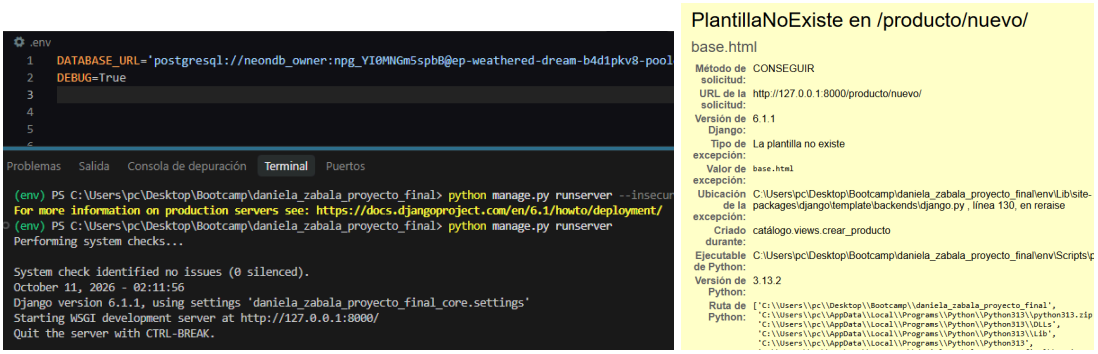
40.Configuración de Debug.

La configuración de Django define DEBUG mediante una variable de entorno. En la versión de entrega, DEBUG debe permanecer desactivado (False). De esta forma, los errores del entorno de despliegue no muestran a los usuarios páginas detalladas con información técnica interna.





Si el Debug se deja activado (True): los errores mostrarán detalles de su procedencia



41. Comprobaciones Finales.

Antes de entregar TechCovers se realizan comprobaciones para verificar que la aplicación arranca, que Django no detecta problemas de configuración y que las pruebas automáticas finalizan correctamente.

También se comprueban manualmente las operaciones principales.

- Ejecutar *python manage.py check* y comprobar que no aparecen problemas de configuración.
- Ejecutar *python manage.py test catalogo --keepdb* y conservar una captura del resultado.
- Probar desde el navegador el registro, el inicio de sesión, el catálogo y la gestión de reservas.
- Comprobar los permisos de administración y que las operaciones de reserva mantienen el stock coherente.
- Verificar que DEBUG está desactivado en la configuración de entrega.

42. Recorrido por el funcionamiento final de TechCovers

En este último apartado se recoge el funcionamiento final de TechCovers mediante un recorrido por las principales funcionalidades de la aplicación.

El objetivo es mostrar el resultado obtenido tras el desarrollo del proyecto y documentar, mediante capturas de pantalla, la navegación, la gestión de usuarios, el catálogo de productos y las operaciones relacionadas con las reservas.



TECHCOVERS

42.1. Página de inicio y navegación

Al acceder a TechCovers, se muestra la página de inicio, que presenta la identidad visual de la tienda y su propuesta.

La plantilla común incorpora la cabecera, el menú de navegación y el pie de página, manteniendo una estructura visual coherente entre las distintas secciones.

Desde el menú principal se puede acceder al inicio, al catálogo y a la página de contacto. La cabecera también muestra las opciones correspondientes al estado de la sesión del usuario.





42.2. Catálogo de Productos

La sección de catálogo permite consultar los accesorios disponibles en TechCovers. Los productos muestran su información correspondiente, como el nombre, la descripción, el precio, la categoría y las existencias, según los datos registrados.

Esta sección permite al usuario explorar los productos y acceder a las acciones disponibles para cada uno de ellos. Las operaciones que se pueden realizar dependen de los permisos del usuario y de la funcionalidad habilitada en la aplicación.



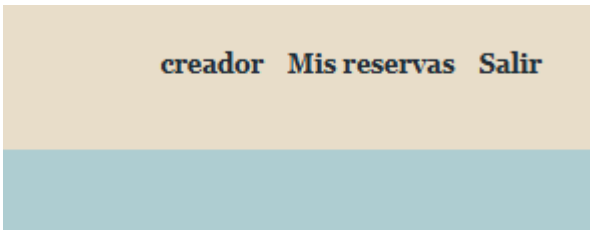


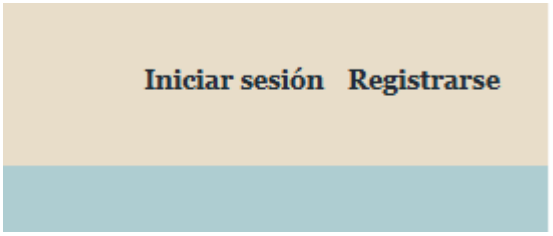
42.3. Registro e inicio de sesión

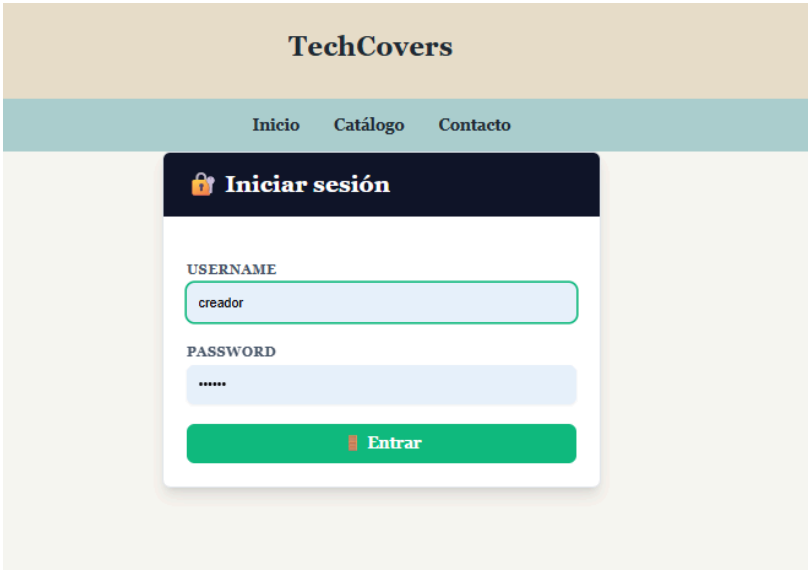
TechCovers incorpora un sistema de usuarios que permite registrarse e iniciar sesión.

Una vez autenticado, el usuario puede acceder a las funcionalidades asociadas a su cuenta.

La aplicación diferencia las opciones de navegación según si el usuario ha iniciado sesión. Cuando la sesión está activa, la cabecera muestra su nombre de usuario y permite acceder a sus reservas o cerrar sesión. Si no está autenticado, se muestran las opciones de inicio de sesión y registro.







42.4. Gestión de reservas

La aplicación permite gestionar las reservas de productos mediante diferentes pantallas. Desde la ficha de un producto se puede iniciar una reserva, indicando la cantidad deseada. También existen pantallas específicas para modificar una reserva y para confirmar su eliminación.

Las operaciones se procesan mediante las vistas de Django, que validan los datos recibidos y controlan las existencias para mantener la coherencia del inventario.



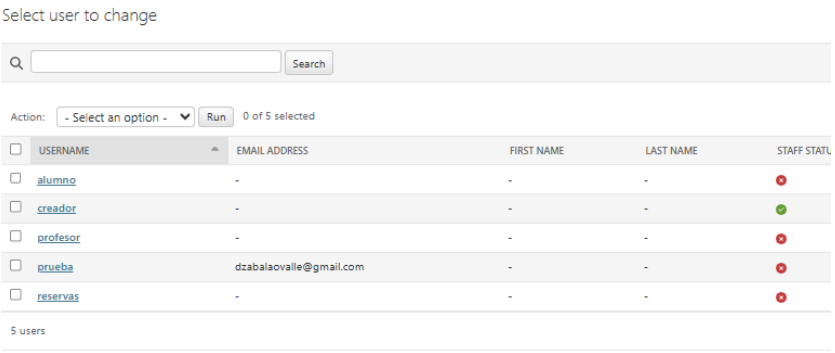
42.5. Administración y permisos

El proyecto utiliza el panel de administración de Django para gestionar los datos registrados en la aplicación.

Desde este panel se pueden consultar los modelos registrados y realizar las operaciones administrativas correspondientes a la configuración del proyecto.

Además, TechCovers diferencia las acciones disponibles según los permisos establecidos. De esta forma, determinadas operaciones de gestión de productos quedan reservadas a los usuarios autorizados.





42.6. Validación y control de errores

Durante el funcionamiento de la aplicación se contemplan situaciones en las que una operación no puede completarse, por ejemplo, cuando los datos introducidos no son válidos o la cantidad solicitada supera las existencias disponibles.

En estos casos, la aplicación utiliza las validaciones y los mecanismos de gestión de errores implementados en el proyecto para evitar operaciones incorrectas y mostrar los mensajes correspondientes cuando proceda.



42.7. Página de contacto

TechCovers incorpora una página de contacto desde la que el usuario puede consultar la información disponible para comunicarse con la tienda.

Se accede a ella mediante la opción Contacto del menú principal, que se mantiene visible en las distintas páginas de la aplicación.

Esta sección completa la navegación de la web y permite centralizar la información de contacto en una página específica.



42.8. Páginas de error personalizadas

Para mejorar la experiencia del usuario, TechCovers incorpora páginas de error personalizadas para los errores HTTP 403, 404 y 500. Estas páginas permiten informar de los problemas que pueden producirse durante la navegación y mantener una presentación visual coherente con el resto de la aplicación.

La página 403 (Forbidden) informa de que el usuario no tiene autorización para acceder a un recurso o realizar una acción determinada. Este error está relacionado con los permisos de acceso y permite diferenciar una operación no autorizada de otros problemas de navegación.

La página 404 (Not Found) se muestra cuando se solicita una dirección que no existe o no se encuentra disponible. En lugar de mostrar una respuesta genérica del servidor, presenta un mensaje adaptado al diseño de TechCovers.

La página 500 (Internal Server Error) está destinada a los errores internos que se producen durante el procesamiento de una solicitud. Su finalidad es informar al usuario de que ha ocurrido un problema sin mostrar detalles técnicos internos de la aplicación.

En conjunto, estas páginas contribuyen a mejorar la experiencia de navegación y permiten gestionar distintas situaciones de error de forma más clara para el usuario. Su funcionamiento depende de la configuración de Django y del entorno en el que se ejecute la aplicación.



43. Mejoras Futuras TechCovers

Como posibles mejoras futuras de TechCovers, se plantean las siguientes:

- **Ampliación del catálogo visual:** añadir más fotografías de los productos para mostrar mejor sus características.
- **Mejora del diseño:** incorporar efectos visuales y animaciones para hacer la navegación más atractiva.
- **Proceso de compra completo:** implementar el pago online, la gestión de los envíos y las devoluciones de productos.